

Salman Adnan Site

Deep Dive: every folder, every file, every line

Digitalise Agency

2026-08-11

Contents

1	What this project is	2
1.1	The problem it was built to solve	2
1.2	The split between this repository and the live site	2
1.3	Reading order	3
1.4	A note on two characters	3
2	The ten-minute mental model	4
2.1	The one paragraph that matters	4
3	Background a beginner needs	5
3.1	What happens when someone opens the page	5
3.2	Tags, attributes, classes and selectors	5
3.3	Custom properties: values named once	6
4	Repository layout	6
5	Architecture	7
6	Data flow, end to end	8
7	The file-by-file walkthrough	8
7.1	new-design-reference/index.html	8
7.2	new-design-reference/css/site.css	24
7.3	new-design-reference/js/main.js	45
7.4	new-design-reference/netlify.toml	52
7.5	new-design-reference/assets/ASSETS.md	54
7.6	.github/workflows-disabled/deploy.yml.disabled	57
8	Generated and third-party files	64
9	Configuration and secrets	64
9.1	Secrets	64
10	Dependencies	65
11	How to run it	65
11.1	What you should see	65
12	The agent-facing surface	66

13 Deployment	66
13.1 What would happen if it were restored	66
14 Honest findings	67
14.1 The design was never finished	67
14.2 The rebrand to orange was left half done	67
14.3 Dead code	68
14.4 Content and consistency	68
14.5 Accessibility	68
15 The project's story	68
15.1 Deliverables	69
15.2 Timeline and effort	69
15.3 Challenges	69
15.4 Outcomes	70
15.5 What to do (judgement)	70
15.6 What not to do (judgement)	70
15.7 If you were starting over (judgement)	71
15.8 Open threads	71
16 Glossary	71
A Appendix A: the file ledger	73

1 What this project is

This repository holds a personal brand website for **salmanadnan.com**: a single page introducing Salman Adnan, showing six client projects, and asking the visitor to book a thirty-minute call.

It is six files and 1,180 lines. No framework, no build step, no package manager, no database, no server-side code. A browser can open the page straight off a disk.

The one thing to understand before anything else

This repository does *not* contain the page currently served at salmanadnan.com. The design here was finished, then deliberately parked, and the machinery that published it was switched off in the same commit. Something else serves that address today. Section 1.2 has the evidence.

Its twin, and why you should read both

There is a near-identical sibling repository in this workspace, **ai-digitalise**, built in the same week from the same skeleton. It has exactly the same six file paths, the same three-file architecture, the same publishing workflow and the same fate.

The two are not copies: every substantive file differs. This one is a personal brand page in orange; that one is an agency service page in blue. Where a difference between them is instructive, this document points it out, because comparing the pair teaches more than reading either alone.

1.1 The problem it was built to solve

A consultant needs a page to send a prospective client to. It has to say who he is, prove he has done this before, and make booking a conversation one click. That is the whole purpose.

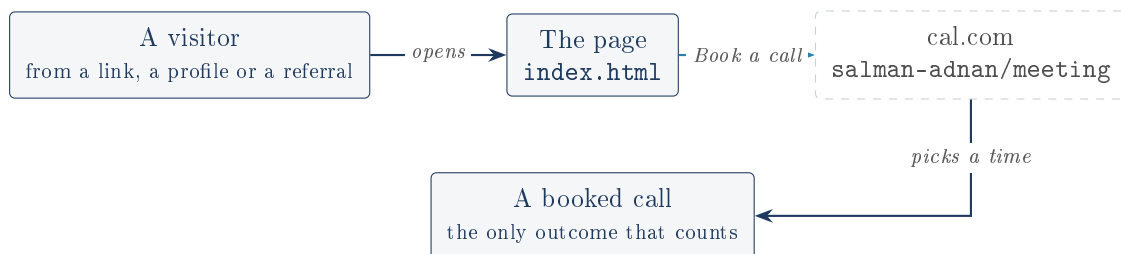


Figure 1: The whole project. The dashed box is a third-party service. Everything in these six files exists to move a visitor from the left box to the bottom box.

1.2 The split between this repository and the live site

The final commit, `0dd839e` on 2026-07-29, moved every file down into a folder called **new-design-reference/** and renamed the publishing workflow so it could no longer run. Its message says why:

“The live site is served from the portfolio repo, not this one. Parking this design for reference and turning the workflow off so it cannot overwrite the real site.”

Checked against the live address on 2026-08-11, while this document was being written:

What was checked	What came back
https://salmanadnan.com/	200, the site is up
http://salmanadnan.com/	308, redirects to the secure version
Address it resolves to	46.202.194.85, the VPS named in the workflow
Title served	<i>Salman Adnan then AI & Full-Stack Engineer</i>
Title in this repository	<i>Salman Adnan: AI Systems for Entrepreneurs</i>
Markers unique to this repository	none present in the live page
Booking link on the live page	cal.com/salman-adnan/meeting , the same one

Table 1: Verified by fetching the live address on 2026-08-11. The live page is a different build that happens to share this project’s booking link.

This repository holds the answer to a bug in its sibling

The `ai-digitalise` Deep Dive records a finding it could not fully explain: the page live at `ai.digitalise.agency` declares its canonical URL to be <https://salmanadnan.com/>, which tells search engines to index this domain instead of that one.

Fetching both addresses on 2026-08-11 completes the picture. The live `salmanadnan.com` declares its own canonical correctly and carries a proper `og:image`; the live `ai.digitalise.agency` declares `salmanadnan.com`’s. Both are served from the same VPS. So the agency page was produced from this personal site’s build and its canonical link was never changed.

Neither repository can fix that, because neither contains the live build. It is recorded in both documents so that whoever maintains the portfolio repository finds it.

Jargon: Canonical URL

A line in a page’s head saying “the authoritative home of this content is at this address”. Search engines use it to avoid indexing the same content twice. Pointing it at the wrong domain hands your search ranking to that domain.

1.3 Reading order

Files are explained in the order that makes each understandable when you reach it, not alphabetically:

1. `new-design-reference/index.html`, the page itself, which every other file serves.
2. `new-design-reference/css/site.css`, which the page names and cannot be read without.
3. `new-design-reference/js/main.js`, which animates what the first two have already drawn.
4. `new-design-reference/netlify.toml`, a setting for a host never used.
5. `new-design-reference/assets/ASSETS.md`, a checklist never completed.
6. `.github/workflows-disabled/deploy.yml.disabled`, about publishing, which only makes sense once you know what is published.

1.4 A note on two characters

Two characters in this repository’s sources cannot be printed by this document’s typeface. They are printed as the closest stand-in, listed so no quoted line misleads you:

In the real file	Printed here as	Where it appears
A long horizontal rule (U+2500)	a hyphen	comment banners in the CSS and JavaScript
A double horizontal rule (U+2550)	an equals sign	comment banners in the HTML

Table 2: Character substitutions used in every quoted listing in this document.

The middle dot (U+00B7) prints exactly and is not substituted.

2 The ten-minute mental model

A web page is made of three kinds of file, and this project has one of each.

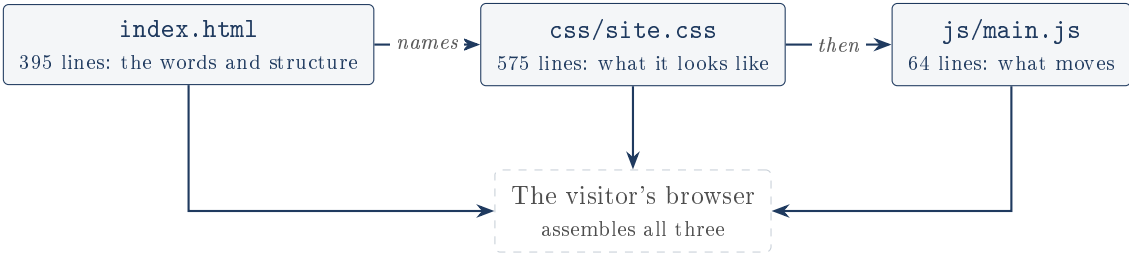


Figure 2: Nothing runs on a server. Everything happens on the visitor’s machine.

Jargon: HTML

HyperText Markup Language: the file holding the page’s actual content, wrapped in labels called tags that say what each piece of content *is*. A tag looks like `<h1>` to open and `</h1>` to close.

Jargon: CSS

Cascading Style Sheets: the file deciding appearance. It names a kind of element (“every heading”, “anything labelled `btn`”) and lists the rules that apply. “Cascading” means there is a defined pecking order when two rules disagree.

Jargon: JavaScript

A programming language running inside the browser after the page loads. It is what makes a page react to a scroll or a click.

2.1 The one paragraph that matters

`index.html` stacks six bands: a fixed navigation bar, a two-column hero with a large photo on the left and the name, tagline and three small statistics on the right, a grid of six client projects, a band of four large counting numbers, two case-study cards, a booking panel and a footer. `site.css` gives all of it a near-black background with an orange accent, and rearranges it into one column below 960 pixels. `main.js` does three things: the navigation bar gains a frosted background past 40 pixels of scroll, the four large numbers count up from zero when they scroll into view, and most blocks fade upward as they arrive. Every “Book a call” button is a link: two jump down the page, and the one in the booking panel opens a `cal.com` calendar in a new tab.

The hero photo is a placeholder, and it says so on screen

The left half of the hero, which is half the visible page when it loads, contains no photograph. It contains a large letter **S** over an orange gradient and the words “*Drop Salman’s photo here / assets/salman.jpg*”.

That text is not a code comment. It is content, styled and positioned, and any visitor would read it. This is the single most consequential fact about the design’s state, and it is examined in section 7.1.

3 Background a beginner needs

3.1 What happens when someone opens the page

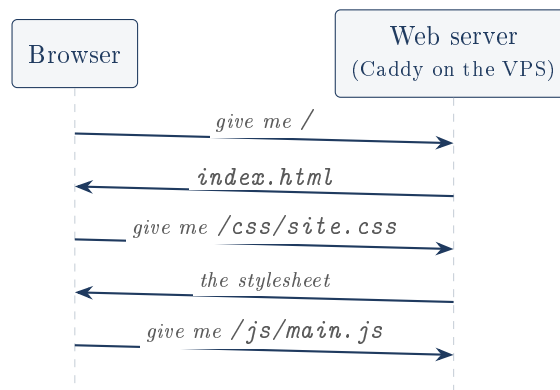


Figure 3: The browser reads `index.html` first and discovers the other two files named inside it. The `<link>` and `<script>` tags are the only reason it knows they exist.

Jargon: Web server

A program on an always-on computer that waits for requests for files and sends them back. This project is served by Caddy on the agency’s rented machine, which hands over files exactly as they sit on disk.

Jargon: VPS

Virtual Private Server: a computer rented from a hosting company and controlled remotely. The workflow names its address as `46.202.194.85`, which is the same address `salmanadnan.com` resolves to today.

3.2 Tags, attributes, classes and selectors

A **tag** marks a piece of content: `<p>` a paragraph, `<section>` a band of the page, `<a>` a link. Most come in pairs with content between.

An **attribute** is extra information inside the opening tag, written as a name and a value: in `<a href="#book">`, `href` has the value `#book`, meaning “the thing named `book` on this same page”.

A **class** is a label you invent and attach with the `class` attribute. It means nothing to the browser; it exists so CSS can find those elements.

A **selector** is how CSS names what it wants: `.btn` means “every element labelled `btn`”.

Jargon: Element

One thing on the page, created by a tag: a heading, a button, a box.

3.3 Custom properties: values named once

The stylesheet opens with a block giving names to values: `-accent: #ff6719;` means “from now on `-accent` stands for this orange”. The rest of the file writes `var(-accent)`.

Jargon: Hex colour

A colour written as a hash and six characters, three pairs giving red, green and blue from 00 to ff. `#ff6719` is a strong orange: full red, a little green, almost no blue.

Jargon: Custom property, also called a CSS variable

A named value defined in CSS with two leading hyphens and read back with `var(...)`. This project defines nineteen and reads fifteen of them.

4 Repository layout

```

Salman Adnan Site
├── .github
│   └── workflows-disabled
│       └── deploy.yml.disabled
└── new-design-reference
    ├── assets
    │   └── ASSETS.md
    ├── css
    │   └── site.css
    ├── js
    │   └── main.js
    ├── index.html
    └── netlify.toml
  
```

Figure 4: The complete repository. Nothing is folded away: these six files are everything git tracks. Appendix A carries the same list with line counts.

Two things about this shape are deliberate and both are messages rather than mechanics.

First, **every file lives under `new-design-reference/`**. Normally `index.html` sits at the top so a web server pointed at the folder finds a home page. Here it is one level down, so a server pointed at this folder finds nothing. The folder name is a note to the next person: this is a reference, not a site.

Second, **the automation folder is misspelled on purpose**. GitHub only runs files in a folder named exactly `.github/workflows`. This one is `.github/workflows-disabled`, and the file inside ends `.disabled` rather than `.yml`. Either change alone would stop it. Both were made.

Jargon: Hidden folder

A folder whose name begins with a dot, hidden from ordinary listings on Mac and Linux. The dot is a convention meaning “tooling lives here, not content”.

5 Architecture

There is no architecture in the usual sense, because nothing runs. What exists is a set of one-way relationships between six files.

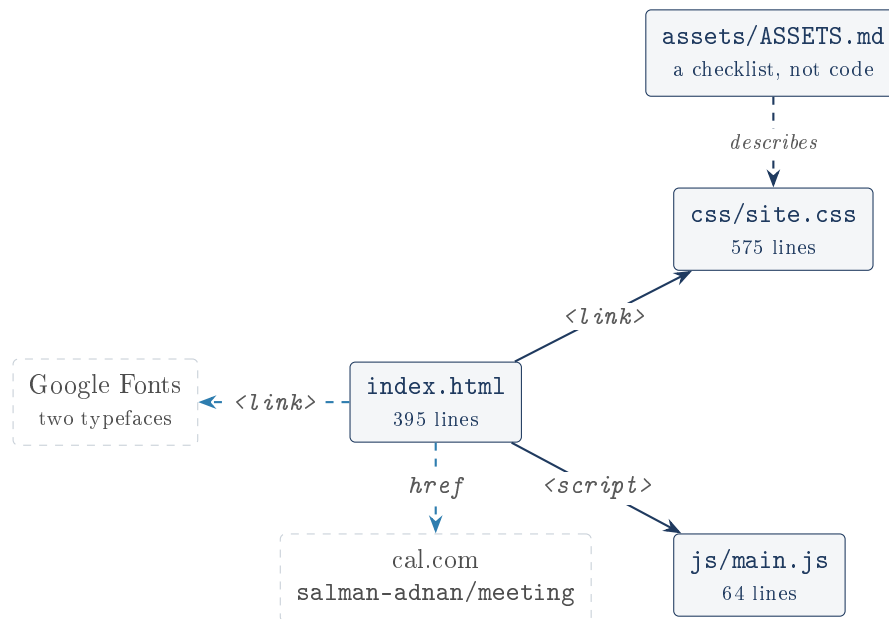


Figure 5: What names what. Dashed boxes are services on other people’s computers. `netlify.toml` and the workflow are absent because nothing on the page refers to them: they instruct the machines that publish the page, not the page.

The arrows go one way only. `index.html` knows the names of the other two files; neither knows anything about it. The stylesheet styles class names it hopes exist, and the JavaScript looks for elements that may not be there, with a guard at every lookup so it does nothing quietly when they are missing.

Silent failure is the price of this simplicity

Nothing checks that the class names in `index.html` and the selectors in `site.css` agree. Section 7.2 documents a case where they do not: a pair of stylesheet rules for the reversed case-study card target a class the HTML never uses, so they have never once applied. The page looks right anyway, because the HTML achieves the same layout by a different means. Nothing anywhere reported it.

6 Data flow, end to end

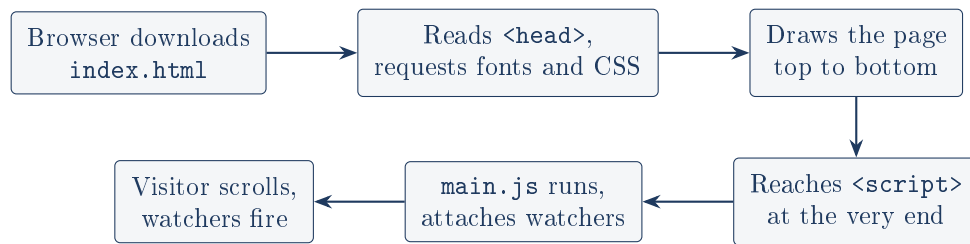


Figure 6: The full life of one page view. The JavaScript runs last because its `<script>` tag is the last thing in the document, which is deliberate and is explained in section 7.3.

There is no data flow beyond this. No form, nothing saved, nothing sent. The only outbound step is following the `cal.com` link, at which point the visitor has left.

Nothing measures whether the page works

No analytics of any kind: no counter, no tag manager, no event when somebody presses “Book a call”. The question this page exists to answer cannot be answered from the page. Notably, the live `salmanadnan.com` *does* carry a full social-preview setup that this version lacks, so whoever built the live version thought about distribution in a way this design did not.

7 The file-by-file walkthrough

Every file git tracks gets its own subsection below, quoted from its first line to its last. Nothing is summarized away and nothing is skipped.

7.1 `new-design-reference/index.html`

HTML, 395 lines. The page itself: every word a visitor reads and the structure holding those words. Delete every other file and this one still opens in a browser and still says everything the project wants to say. It would simply look plain.

Where this came from: `new-design-reference/index.html`

Hand-written. It arrived complete in the first commit, `1f7ed12` on 2026-07-28, at the path `index.html`, and moved to its present path in `0dd839e` on 2026-07-29 with no change to its contents. No generator banner, no framework scaffold, and no `package.json` that could have produced it.

Its icons are a different matter and worth naming precisely. Every icon is an inline SVG whose path data matches the Heroicons outline set, a widely used free library: the shapes, the 24 24 view box, the `stroke-linecap` and `stroke-linejoin` attributes and the 1.5-based coordinates are that library’s house conventions. **The file carries no attribution comment**, so this is an inference from the path data, not a documented fact.

Its skeleton is shared with the sibling repository `ai-digitalise`: identical head block, identical navigation shape, identical metrics band, identical booking panel and footer. Which was written first is not recorded in either repository and is not guessed here; `ai-digitalise` was committed a few hours earlier on the same day. **Confidence: certain for hand-written, inferred for the icon origin, and the sibling relationship is stated as a resemblance with a shared skeleton rather than as a direction of copying.**

Why it exists

Without this file there is no page. The stylesheet would style class names belonging to nothing, and the JavaScript would search for elements that do not exist and quietly do nothing, which is exactly how it is written to behave.

Line by line

Lines 1 to 17: the head, which the visitor never sees

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 <title>Salman Adnan: AI Systems for Entrepreneurs</title>
7 <meta name="description" content="I build AI systems and digital products that grow businesses
  without adding headcount. 17 projects, 1.6M+ organic views." />
8 <meta property="og:title" content="Salman Adnan" />
9 <meta property="og:description" content="AI systems that grow your business without adding
  headcount." />
10 <meta property="og:type" content="website" />
11 <meta name="theme-color" content="#05070d" />
12
13 <link rel="preconnect" href="https://fonts.googleapis.com" />
14 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
15 <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@500;600;700&family=
  Inter:wght@400;500;600;700&display=swap" rel="stylesheet" />
16 <link rel="stylesheet" href="/css/site.css" />
17 </head>
```

Listing 1: index.html, lines 1–17

Line 1 is not a tag. `<!DOCTYPE html>` declares “interpret this as modern HTML”. Without it, browsers fall back to a compatibility mode imitating software from the 1990s. It must come first and has no closing partner.

Line 2 opens the document and declares its language English. That attribute does real work: screen readers use it to choose a pronunciation and search engines use it to decide who to show the page to.

Line 3 opens the `<head>`. Everything to line 17 is information *about* the page rather than content *of* it, and none of it appears on screen.

Line 4 declares the character encoding UTF-8, the scheme letting a file contain any character in any language. It matters concretely here because the page contains a middle dot, which would otherwise render as nonsense. It should come as early as possible, since the browser guesses until told.

Jargon: Meta tag

A tag inside `<head>` carrying a fact about the page rather than content for it. It has no closing partner and displays nothing. Different meta tags are read by completely different audiences: browsers, search engines and social networks each look for their own.

Line 5 is the viewport declaration, and it is the single line making the page work on a phone. `width=device-width` tells the browser to treat the page width as the real device width, and `initial-scale=1.0` says do not zoom out to begin with. Without it a phone pretends to be about 980 pixels wide and shrinks everything, which is why old sites look like tiny unreadable newspapers. Every media query in the stylesheet depends on this line.

Line 6 is the title: the browser tab text and the blue headline in a search result. Note it, because section 1.2 used it to prove this page is not the live one.

Line 7 is the description search engines often show as grey text under the title. It is worth reading closely, because it makes two specific claims (“17 projects, 1.6M+ organic views”) that also appear on the page itself in two other places.

Lines 8 to 10 are Open Graph tags, a convention invented by Facebook and now read by nearly every chat app to build the preview card when somebody pastes a link.

The Open Graph set is missing the tag that matters most

Present: `og:title`, `og:description`, `og:type`. Missing: `og:image` and `og:url`. Without an image, a pasted link produces a preview card with no picture, which is the difference between a link that gets clicked and one that does not.

This is more pointed here than in the sibling repository. The live `salmanadnan.com` carries a complete set including `og:image` pointing at `/assets/og.png`, verified on 2026-08-11. So whoever built the live version supplied exactly the asset this one lists as required and never produced. See section 7.5.

Line 11 sets `theme-color` to `#05070d`, the same near-black as the page background, so a mobile browser paints its address bar to match and the browser chrome blends into the page instead of framing it in white.

The theme colour was not rebranded

`#05070d` is the shared near-black, which is correct. But note it is identical to the sibling’s, as is the entire head block apart from the title and description. The rebrand described in commit `05210af` touched the stylesheet and left this file’s shared parts alone, which is consistent with the five missed colour literals documented in section 7.2.

Lines 13 and 14 are `preconnect` hints, telling the browser it will shortly need these two Google servers so it should start the handshake now. Establishing a secure connection takes several network round trips, and doing that early typically saves a few hundred milliseconds. The `crossorigin` attribute on line 14 is required: font files are fetched under stricter rules than stylesheets, and a `preconnect` made without it opens a connection the font request cannot reuse. That subtlety catches out experienced developers constantly.

Line 15 fetches the fonts: Space Grotesk in three weights and Inter in four. `display=swap` at the end is the important part. It tells the browser to render text immediately in a fallback typeface and swap in the real one when it arrives, rather than hiding text for up to three seconds. Text reflows slightly; the benefit is that a visitor on a slow connection can read the page at all.

Jargon: Font weight

How heavy a typeface looks, on a numeric scale where 400 is normal and 700 is bold. Requesting `weight@500;600;700` downloads three separate files, which is why the list is kept short.

Line 16 links the stylesheet, with the leading slash discussed in section 11: correct when served by a web server, wrong when the file is opened off a disk.

Line 17 closes the head.

Lines 18 to 28: the navigation bar

```

18 <body>
19
20 <!-- =====
21     NAV
22     ===== -->
23 <nav class="nav" aria-label="Site navigation">
24   <div class="nav-inner">
25     <span class="nav-logo">Salman Adnan</span>

```

```

26     <a href="#book" class="btn btn-primary">Book a call</a>
27   </div>
28 </nav>

```

Listing 2: index.html, lines 18–28

Line 18 opens `<body>`. Everything to line 394 is what a visitor sees.

Lines 20 to 22 are a comment banner. Anything between `<!--` and `-->` is ignored entirely by the browser. This file uses six of these as signposts, one per band, each drawn with a rule of repeated double-line characters. They are the author’s table of contents and the reason a 395-line file stays navigable.

Line 23 opens the navigation. `<nav>` is a *semantic* tag: it looks no different from a plain `<div>` but announces to screen readers and search engines that this is the site’s navigation. `aria-label` gives that region a spoken name.

Jargon: ARIA

Accessible Rich Internet Applications: attributes beginning `aria-` that describe a page to assistive software. They change nothing visually. `aria-label` supplies a name; `aria-hidden="true"` tells assistive software to skip something because it is purely decorative.

Line 24 opens a `<div>` labelled `nav-inner`. A `<div>` is a generic container with no meaning, existing purely to be styled. The reason for a second container is a standard layout pattern: the outer element spans the full window so its background stretches edge to edge, while the inner one is capped and centred so the logo and button line up with the content below.

The logo is not a link

Line 25 is a `<span>` containing the words “Salman Adnan”. It is not clickable, so there is no way back to the top of the page by clicking it, which is the behaviour every visitor expects from a site logo.

On a single-page site this is defensible, and it differs from the sibling repository, where the logo is an `<a>` pointing at `/` with a drawn mark beside it. That divergence is one of the few places where the two skeletons genuinely part company, and this is the weaker of the two.

Line 26 is the navigation’s only button, a link to `#book`. It is a link and not a `<button>`, which is correct: it navigates rather than performing an action. It carries two classes, `btn` for the shared shape and `btn-primary` for the filled variant. That two-class pattern repeats throughout and is explained in section 7.2.

Lines 27 and 28 close the containers.

Lines 29 to 46: the hero opens, and the placeholder that defines this project’s state

```

29
30
31 <!-- =====
32     HERO
33     UPDATE: replace .hero-photo-placeholder with:
34     
36     ===== -->
37 <section class="hero" aria-label="Hero">
38
39   <!-- photo side -->
40   <div class="hero-photo-side">
41     <div class="hero-photo-frame">
42
43     <!-- PLACEHOLDER: swap for Salman's actual photo (see comment above) -->

```

```

43     <div class="hero-photo-placeholder" aria-hidden="true">
44         <div class="placeholder-initial">S</div>
45         <div class="placeholder-note">Drop Salman's photo here<br>assets/salman.jpg</div>
46     </div>

```

Listing 3: index.html, lines 29–46

Lines 31 to 35 are the second banner, and it carries an instruction rather than just a label: replace the placeholder with an `<img>` tag, given in full, including the exact styling needed (`object-fit: cover` to fill the frame without distortion and `object-position: top` to keep a face in shot when the frame is cropped). Whoever wrote this knew precisely what was needed and did not have the file.

Line 36 opens the hero. Line 39 opens the left column and line 40 the frame that holds the photograph and the badges floating over it.

Lines 42 to 46 are the most consequential nine lines in the repository.

Half the opening screen is a placeholder addressed to the developer

The left column of the hero, which on a desktop is half of everything visible when the page loads, contains no photograph. It contains a giant letter **S** in an orange gradient and, beneath it, the words “*Drop Salman’s photo here*” followed by “*assets/salman.jpg*”.

That is not a code comment. It is content: it sits inside the page, is styled by `site.css` lines 173 to 179, and renders on screen. The `aria-hidden` on line 43 hides it from screen readers, which is correct for a decorative gradient and means a screen-reader user is spared the instruction, while every sighted visitor reads it.

For a personal brand page, whose entire proposition is a person, the missing element is the person. This single fact is why the honest description of this project is “unfinished” rather than “finished and parked”. The sibling `ai-digitalise` has no equivalent: its placeholders are gradients that merely look like unfinished thumbnails, not text asking for a file.

Note that the note names the path `assets/salman.jpg` while the instruction in the banner comment above names `assets/salman.jpg` and `ASSETS.md` names `/assets/salman.jpg` with a leading slash. The leading slash is the one that would actually work, for the same reason as the stylesheet path.

Lines 47 to 59: the first floating badge

```

47
48     <!-- floating result badges over the photo -->
49     <div class="hero-float-badge" aria-hidden="true">
50         <div class="badge-icon badge-icon-blue">
51             <svg viewBox="0 0 24 24" fill="none" stroke="#3f82ff" stroke-width="2" aria-hidden="
52             true">
53                 <path stroke-linecap="round" stroke-linejoin="round" d="M3 13.125C3 12.504 3.504
54                 12 4.125 12h2.25c.621 0 1.125.504 1.125 1.125v6.75c7.5 20.496 6.996 21 6.375 21h-2.25A1
55                 .125 1.125 0 013 19.875v-6.75zM9.75 8.625c0-.621.504-1.125 1.125-1.125h2.25c.621 0
56                 1.125.504 1.125 1.125v11.25c0 .621-.504 1.125-1.125 1.125h-2.25a1.125 1.125 0
57                 01-1.125-1.125V8.625zM16.5 4.125c0-.621.504-1.125 1.125-1.125h2.25C20.496 3 21 3.504 21
58                 4.125v15.75c0 .621-.504 1.125-1.125 1.125h-2.25a1.125 1.125 0 01-1.125-1.125V4.125z"/>
59             </svg>
60         </div>
61         <div>
62             <div class="badge-text-main">17 projects</div>
63             <div class="badge-text-sub">delivered</div>
64         </div>
65     </div>

```

Listing 4: index.html, lines 47–59

Three small frosted cards drift slowly over the photograph. Each has an icon in a tinted square and two lines of text: a number and a caption.

Line 49 is the container, marked `aria-hidden` because the same claims appear as real text further down the page. That is the correct call: repeating them to a screen reader would be duplication.

Lines 51 to 53 draw a bar-chart icon. Note `stroke="#3f82ff"`, a literal blue, rather than a token.

The badge icons are the one place blue is deliberate

Three badges use three different colours: blue here, orange on line 63, green on line 75, each matching a tinted background defined at `site.css` lines 230 to 232. Used as a set of three distinct signal colours this is a reasonable design choice, and it is the only appearance of blue in this file that is plainly intentional rather than left over from the sibling.

It is still written as three literals where two tokens exist (`-accent2` is exactly this blue and is otherwise never used, and `-green` is exactly that green). Section 7.2 covers what that costs.

Lines 55 to 58 hold the text: “17 projects” and “delivered”. That number will appear twice more before the page ends.

Lines 60 to 71: the second badge

```

60
61     <div class="hero-float-badge" aria-hidden="true">
62         <div class="badge-icon badge-icon-orange">
63             <svg viewBox="0 0 24 24" fill="none" stroke="#ff6719" stroke-width="2" aria-hidden="
64             true">
65                 <path stroke-linecap="round" stroke-linejoin="round" d="M2.25 18L9 11.25L4.306
66                 4.307a11.95 11.95 0 015.814-5.519L12.74-1.22m0 0l-5.94-2.28m5.94 2.28l-2.28 5.94l"/>
67             </svg>
68         </div>
69         <div>
70             <div class="badge-text-main">1.6M+ views</div>
71             <div class="badge-text-sub">organic, for clients</div>
72         </div>

```

Listing 5: `index.html`, lines 60–71

Structurally identical, with an upward-trending chart icon in orange and the claim “1.6M+ views / organic, for clients”. The orange `#ff6719` here is the brand accent written literally rather than as `var(-accent)`.

Lines 72 to 86: the third badge, and the end of the photo column

```

72
73     <div class="hero-float-badge" aria-hidden="true">
74         <div class="badge-icon badge-icon-green">
75             <svg viewBox="0 0 24 24" fill="none" stroke="#34d399" stroke-width="2" aria-hidden="
76             true">
77                 <path stroke-linecap="round" stroke-linejoin="round" d="M9 12.75L11.25 15 15 9.75
78                 M21 12a9 9 0 11-18 0 9 9 0 0118 0z"/>
79             </svg>
80         </div>
81         <div>
82             <div class="badge-text-main">300% avg growth</div>
83             <div class="badge-text-sub">across client sites</div>
84         </div>
85     </div>
86 </div><!-- /photo side -->

```

Listing 6: index.html, lines 72–86

The third badge uses the tick-in-a-circle icon and the claim “300% avg growth / across client sites”. Its green is #34d399, which is exactly the value of the `-green` token defined at `site.css` line 15 and written literally here.

Line 85 closes the frame and line 86 closes the column, with a trailing comment naming what was closed. That habit is worth copying: in a file with this much nesting a bare `</div>` tells the reader nothing.

Why these three badges are numbered from 2 in the stylesheet

The stylesheet positions the badges with `.hero-float-badge:nth-child(2)`, (3) and (4), at `site.css` lines 217 to 219. They are children 2, 3 and 4 of the frame because the placeholder on line 43 is child 1.

This survives the intended fix: replace the placeholder with an `<img>` as the comment instructs and the image becomes child 1, leaving the badges where they are. That is either careful thinking or luck; either way the numbers do not need to change when the photograph arrives, which is worth knowing before somebody “corrects” them.

Lines 87 to 112: the right column, down to the buttons

```

87
88  <!-- content side -->
89  <div class="hero-content-side">
90    <div class="hero-tag">
91      <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
92        width="2.5" aria-hidden="true">
93        <path stroke-linecap="round" stroke-linejoin="round" d="M9.813 15.904L9 18.751
94        -.813-2.846a4.5 4.5 0 00-3.09-3.09L2.25 12l2.846-.813a4.5 4.5 0 003.09-3.09L9 5.25l.813
95        2.846a4.5 4.5 0 003.09 3.09L15.75 12l-2.846.813a4.5 4.5 0 00-3.09 3.09z"/>
96      </svg>
97      Digitalise Agency
98    </div>
99
100    <h1 class="hero-h1">
101      Salman<br>
102      <em>Adnan.</em>
103    </h1>
104
105    <p class="hero-descriptor">I build AI systems that grow<br>your business without adding
106    headcount.</p>
107
108    <div class="hero-actions">
109      <a href="#book" class="btn btn-primary">
110        Book a call
111        <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
112        stroke-width="2.5" aria-hidden="true">
113          <path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l-7.5 7.5
114          M21 12H3"/>
115        </svg>
116      </a>
117      <a href="#work" class="btn btn-ghost">See the work</a>
118    </div>

```

Listing 7: index.html, lines 87–112

Line 89 opens the content column. Lines 90 to 95 are the small pill above the name, containing a sparkle icon and the words “Digitalise Agency”, which ties this personal page back to the agency.

Lines 97 to 100 are the headline in an `<h1>`. A page should have exactly one, and this page does: it is the document’s title as far as a screen reader or a search engine is concerned, and here

it is simply the person’s name. `<br>` forces the line break so the surname lands on its own line.

`<em>` is used for colour, not for emphasis

Line 99 wraps “Adnan.” in `<em>`, and `site.css` line 272 then removes the italics that tag normally implies and colours the word orange instead. `<em>` means emphasis and a screen reader may change its intonation for it. Using it as a colour hook works and slightly misrepresents the content to assistive software; a `<span>` with a class would have been the honest choice. The sibling repository does exactly the same thing, so it is a habit rather than a slip. The intent is plainly visual.

Line 102 is the tagline, and it repeats the page description from line 7 almost word for word. `<br>` again controls where it breaks.

Lines 104 to 112 hold the two buttons: the primary one to `#book` containing both text and an arrow icon, and the ghost one to `#work`. Both are ordinary links, and both are intercepted by `main.js` line 55 to scroll smoothly rather than jump.

Lines 113 to 130: the three small statistics, and the end of the hero

```

113
114     <div class="hero-mini-stats">
115       <div class="hero-mini-stat">
116         <div class="hero-mini-num">17+</div>
117         <div class="hero-mini-lbl">Projects delivered</div>
118       </div>
119       <div class="hero-mini-stat">
120         <div class="hero-mini-num">1.6M+</div>
121         <div class="hero-mini-lbl">Organic views</div>
122       </div>
123       <div class="hero-mini-stat">
124         <div class="hero-mini-num">10K+</div>
125         <div class="hero-mini-lbl">Users served</div>
126       </div>
127     </div>
128   </div>
129
130 </section>

```

Listing 8: `index.html`, lines 113–130

Three statistics under a hairline: 17+ projects, 1.6M+ organic views, 10K+ users served.

The same three numbers exist twice on this page, in two different formats

These three are written as plain visible text. The metrics band at lines 258 to 279 states the same three (plus a fourth) as `data-target` attributes with a visible 0, to be counted up by JavaScript.

So updating a number means editing it in two places, in two different ways, and nothing checks that they agree. `ASSETS.md` line 24 knows this and says so explicitly, which means it was a recognised cost rather than an oversight.

There is an upside worth stating. Because the hero copies are plain text, they are the only version a search engine or a visitor without JavaScript ever sees, so this duplication accidentally rescues the page from the “claims zero of everything” failure that the sibling repository has in full. The sibling has no hero statistics, so its numbers vanish entirely without JavaScript.

Line 130 closes the hero section.

Lines 131 to 162: the work grid, and the first project card

```

131
132
133 <!-- =====
134     WORK PORTFOLIO
135     UPDATE: replace gradient .wt* classes with real thumbnails.
136           Add poster="assets/thumb-N.jpg" to each tile img.
137     ===== -->
138 <section class="work-section" id="work" aria-label="Work portfolio">
139   <div class="container">
140     <div class="reveal" style="margin-bottom:0;">
141       <p class="section-kicker">The work</p>
142       <h2 class="section-title">What I've built<br>for clients.</h2>
143     </div>
144
145     <div class="work-grid stagger">
146
147       <div class="work-card" style="--i:0">
148         <div class="work-thumb wt1">
149           <div class="work-badge">300% traffic</div>
150           <div class="work-thumb-overlay">
151             <div class="work-play">
152               <svg viewBox="0 0 24 24" fill="white" aria-hidden="true">
153                 <path d="M5.25 5.653c0-.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010
154                 1.971l-11.54 6.347a1.125 1.125 0 01-1.667-.985V5.653z"/>
155               </svg>
156             </div>
157           </div>
158           <div class="work-info">
159             <div class="work-client">Hi-Tec Bearings</div>
160             <div class="work-title">5,000+ SKUs, 300% organic traffic growth, 45% faster page
161             loads</div>
162           </div>
163         </div>

```

Listing 9: index.html, lines 131–162

Lines 133 to 137 are the third banner, again carrying an instruction: replace the gradient classes with real thumbnails.

Line 138 carries `id="work"`, which is what makes the hero's "See the work" button work. An `id` names one element, and a link to `#work` scrolls to whatever carries it.

Jargon: id, and how it differs from class

An `id` names exactly one element and must be unique. A `class` labels any number. Use an `id` when something needs to be linked to; use a `class` when you want to style a group. This page has exactly two `ids`, `work` and `book`, and both exist to be jumped to.

Lines 140 to 143 establish a pattern used by the remaining sections: a wrapper with the class `reveal` holding a small uppercase kicker and a large heading. The `reveal` class is what `main.js` line 50 watches for, fading the pair upward when they scroll into view. The inline `margin-bottom:0` on line 140 overrides the 52-pixel margin the stylesheet gives `.section-title`, because this section wants its grid closer than the others do.

Line 142 is an `<h2>`. Heading levels form the document's outline: the single `<h1>` in the hero, then an `<h2>` per major section. Getting that order right is how a screen-reader user navigates a long page, and this file gets it right.

Line 145 gives the grid the class `stagger`, and each card carries `style="--i:0"` and so on. That custom property is multiplied by 90 milliseconds at `site.css` line 86 to delay each child's fade-in, so the cards appear in sequence rather than together. Setting a custom property inline is the

cleanest way to pass a per-element number to CSS.

Each card is two halves. The top, `work-thumb`, is where a real thumbnail would go and is currently a gradient from the class `wt1`. Over it sit a badge and an overlay containing a play button, which the stylesheet keeps invisible until the mouse is over the card. The bottom, `work-info`, holds the client name and the result.

The play buttons promise videos that do not exist

Every card shows a play button on hover and none plays anything: the card is not a link and has no click handler.

This is milder than the same fault in the sibling repository, which also sets `cursor: pointer` so the mouse changes to a hand. Here it does not, so the promise is visual only. The cheapest honest fix in both is to remove the play overlay until the videos exist.

Lines 163 to 196: project cards two and three

```

163
164     <div class="work-card" style="--i:1">
165         <div class="work-thumb wt2">
166             <div class="work-badge">3x leads</div>
167             <div class="work-thumb-overlay">
168                 <div class="work-play">
169                     <svg viewBox="0 0 24 24" fill="white" aria-hidden="true">
170                         <path d="M5.25 5.653c0-.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010
171                         1.971l-11.54 6.347a1.125 1.125 0 01-1.667-.985V5.653z"/>
172                     </svg>
173                 </div>
174             </div>
175             <div class="work-info">
176                 <div class="work-client">DigiMarvixx</div>
177                 <div class="work-title">3x WhatsApp leads per month, 60% mobile traffic capture</div>
178             </div>
179         </div>
180
181     <div class="work-card" style="--i:2">
182         <div class="work-thumb wt3">
183             <div class="work-badge">1.2s load</div>
184             <div class="work-thumb-overlay">
185                 <div class="work-play">
186                     <svg viewBox="0 0 24 24" fill="white" aria-hidden="true">
187                         <path d="M5.25 5.653c0-.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010
188                         1.971l-11.54 6.347a1.125 1.125 0 01-1.667-.985V5.653z"/>
189                     </svg>
190                 </div>
191             </div>
192             <div class="work-info">
193                 <div class="work-client">Synergy Event</div>
194                 <div class="work-title">800 participants, 1.2s load, 98/100 Lighthouse, 100% mobile
195                 score</div>
196             </div>
197         </div>

```

Listing 10: `index.html`, lines 163–196

Structurally identical to the first, differing in the stagger index, the gradient class, the badge text, the client and the result. The six clients across this grid are Hi-Tec Bearings, DigiMarvixx, Synergy Event, IBA GPT, Prestige Sleep and IBA CED, the same six as in the sibling repository, described here in slightly more detail (this version adds “45% faster page loads” and “100% mobile score”).

These are specific, named, checkable claims, which is a meaningfully different kind of statement from the floating badges above them.

Lines 197 to 251: project cards four, five and six

```

197
198     <div class="work-card" style="--i:3">
199         <div class="work-thumb wt4">
200             <div class="work-badge">AI chat</div>
201             <div class="work-thumb-overlay">
202                 <div class="work-play">
203                     <svg viewBox="0 0 24 24" fill="white" aria-hidden="true">
204                         <path d="M5.25 5.653c0-.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010
205                         1.971l-11.54 6.347a1.125 1.125 0 01-1.667-.985V5.653z"/>
206                     </svg>
207                 </div>
208             </div>
209         <div class="work-info">
210             <div class="work-client">IBA GPT</div>
211             <div class="work-title">AI university assistant answering student questions around
the clock</div>
212         </div>
213     </div>
214
215     <div class="work-card" style="--i:4">
216         <div class="work-thumb wt5">
217             <div class="work-badge">24/7 sales</div>
218             <div class="work-thumb-overlay">
219                 <div class="work-play">
220                     <svg viewBox="0 0 24 24" fill="white" aria-hidden="true">
221                         <path d="M5.25 5.653c0-.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010
222                         1.971l-11.54 6.347a1.125 1.125 0 01-1.667-.985V5.653z"/>
223                     </svg>
224                 </div>
225             </div>
226         <div class="work-info">
227             <div class="work-client">Prestige Sleep</div>
228             <div class="work-title">Clinic booking and product shop combined, sells after hours<
/div>
229         </div>
230     </div>
231
232     <div class="work-card" style="--i:5">
233         <div class="work-thumb wt6">
234             <div class="work-badge">4 dashboards</div>
235             <div class="work-thumb-overlay">
236                 <div class="work-play">
237                     <svg viewBox="0 0 24 24" fill="white" aria-hidden="true">
238                         <path d="M5.25 5.653c0-.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010
239                         1.971l-11.54 6.347a1.125 1.125 0 01-1.667-.985V5.653z"/>
240                     </svg>
241                 </div>
242             </div>
243         <div class="work-info">
244             <div class="work-client">IBA CED</div>
245             <div class="work-title">Analytics platform with 4 executive dashboards tracking
entrepreneurship outcomes</div>
246         </div>
247     </div>
248
249 </div>
250 </div>

```

```
251 </section>
```

Listing 11: index.html, lines 197–251

The final three complete the set, with indices 3, 4 and 5 and gradients `wt4` through `wt6`. Two of the six are for the same institution (IBA GPT and IBA CED), the only repetition among them.

Note that the same play-triangle path data appears six times in this section and nowhere is it shared. That is the cost of having no templating system, and it is also the correct trade for a project that deliberately has no build step.

Line 249 closes the grid and lines 250 and 251 the container and section.

Lines 252 to 279: the metrics band

```
252
253
254 <!-- =====
255     METRICS
256     UPDATE: replace data-target values with real numbers
257     ===== -->
258 <section class="metrics-section" aria-label="Results">
259   <div class="container">
260     <div class="metrics-grid stagger">
261       <div class="metric-item" style="--i:0">
262         <div class="metric-num" data-target="17" data-suffix="+">0</div>
263         <div class="metric-label">Projects Delivered</div>
264       </div>
265       <div class="metric-item" style="--i:1">
266         <div class="metric-num" data-target="1.6" data-suffix="M+">0</div>
267         <div class="metric-label">Organic Views for Clients</div>
268       </div>
269       <div class="metric-item" style="--i:2">
270         <div class="metric-num" data-target="10" data-suffix="K+">0</div>
271         <div class="metric-label">Users Served</div>
272       </div>
273       <div class="metric-item" style="--i:3">
274         <div class="metric-num" data-target="300" data-suffix="%">0</div>
275         <div class="metric-label">Avg. Traffic Growth</div>
276       </div>
277     </div>
278   </div>
279 </section>
```

Listing 12: index.html, lines 252–279

This band is where the JavaScript does its most visible work, and it introduces the mechanism the counter depends on.

Look at line 262: the element’s visible content is the single character 0, and the real number lives in `data-target="17"`, with `data-suffix="+"` beside it.

Jargon: Data attribute

Any attribute whose name starts with `data-`. HTML ignores it completely and displays nothing; it exists so JavaScript can attach its own information to an element. `main.js` reads these two at lines 17 and 18 to know what to count towards and what to print after it.

Why is the visible content 0? Because that is what a visitor sees before the counter runs, and it is also what they see forever if JavaScript is disabled or fails. The robust alternative is to write the real number in the HTML and count towards it from zero, which fails gracefully. This file chose the fragile form, and is partly rescued by the hero statistics repeating three of the four as plain text.

This band still carries an `UPDATE` comment on line 256 telling the reader to replace the values with real numbers, even though commit 692e162 claims to have replaced all placeholder content with real project data.

Lines 280 to 318: the case studies, and the first card

```

280
281
282 <!-- =====
283     CASE STUDIES (visual-only, numbers speak)
284     UPDATE: replace gradient backgrounds with real screenshots.
285     Use background-image on .case-visual or swap the div for <img>.
286     ===== -->
287 <section class="cases-section" aria-label="Case studies">
288   <div class="container">
289     <div class="reveal" style="margin-bottom:52px;">
290       <p class="section-kicker">Case studies</p>
291       <h2 class="section-title">Results in numbers.</h2>
292     </div>
293
294     <!-- Case 1 -->
295     <div class="case-card reveal">
296       <div class="case-visual cv1">
297         <div class="case-visual-inner">
298           <div class="case-visual-metrics">
299             <div class="cvm">
300               <span class="cvm-num">300%</span>
301               <span class="cvm-lbl">Traffic growth</span>
302             </div>
303             <div class="cvm">
304               <span class="cvm-num">5K+</span>
305               <span class="cvm-lbl">Products</span>
306             </div>
307             <div class="cvm">
308               <span class="cvm-num">99.9%</span>
309               <span class="cvm-lbl">Uptime</span>
310             </div>
311           </div>
312         </div>
313       </div>
314       <div class="case-content">
315         <div class="case-client-name">Hi-Tec Bearings</div>
316         <div class="case-title">Industrial e-commerce with 5,000+ SKUs that tripled organic
317         traffic</div>
318       </div>
319     </div>

```

Listing 13: index.html, lines 280–318

Two large cards, each a two-column arrangement of a visual panel and a block of text. The visual is a gradient standing in for a screenshot, with three small frosted chips of metrics laid over its bottom edge.

Line 295 gives the card the classes `case-card` and `reveal`, so it fades in as a whole rather than being staggered.

Lines 299 to 310 are the three metric chips: 300% traffic growth, 5K+ products, 99.9% uptime. Note that 99.9% uptime is a claim appearing nowhere else on the page and in no other repository in this pair.

Lines 319 to 347: the second case study, and a dead class

```

319
320 <!-- Case 2 -->
321 <div class="case-card case-card-reverse reveal">
322   <div class="case-visual cv2" style="order:2">

```

```

323     <div class="case-visual-inner">
324         <div class="case-visual-metrics">
325             <div class="cvm">
326                 <span class="cvm-num">3x</span>
327                 <span class="cvm-lbl">Leads/month</span>
328             </div>
329             <div class="cvm">
330                 <span class="cvm-num">60%</span>
331                 <span class="cvm-lbl">Mobile traffic</span>
332             </div>
333             <div class="cvm">
334                 <span class="cvm-num">WhatsApp</span>
335                 <span class="cvm-lbl">Integration</span>
336             </div>
337         </div>
338     </div>
339 </div>
340 <div class="case-content" style="order:1">
341     <div class="case-client-name">DigiMarvixx</div>
342     <div class="case-title">Digital marketing site that tripled monthly leads through
WhatsApp</div>
343 </div>
344 </div>
345
346 </div>
347 </section>

```

Listing 14: index.html, lines 319–347

The second card is meant to mirror the first: text on the left, visual on the right. Line 321 gives it the extra class `case-card-reverse`, and lines 322 and 340 set `order:2` and `order:1` inline.

The reversal class matches nothing, and the layout works anyway

`site.css` lines 436 and 437 define `.case-card.reverse`, which means “an element carrying both the class `case-card` and the class `reverse`”. This element carries `case-card` and `case-card-reverse`. A hyphen does not create a second class, so **those rules have never once applied**, and neither has the third at line 559 in the responsive block.

The reversal happens regardless, because of the inline `order` properties on lines 322 and 340, which tell the grid to place the visual second and the text first. So the design is correct, three stylesheet rules are dead, and nothing anywhere reported it.

This is the clearest example in the pair of what “silent failure” means in practice: a whole mechanism can be wrong while the page looks right, because a second mechanism quietly does the same job.

Jargon: order

A CSS property setting an item’s position inside a flex or grid container independently of where it sits in the HTML. `order:2` places an item after one with `order:1`, regardless of which is written first.

Note also that the inline `style="order:2"` on line 322 sits alongside the class `cv2`, so one element is being positioned by an inline style and coloured by a class, which is the same mixed habit seen throughout the pair.

Lines 348 to 377: the booking panel

```

348
349
350 <!-- =====
351     BOOK A CALL

```

```

352     UPDATE: replace YOUR_CAL_LINK with your Cal.com or Calendly URL.
353     ===== -->
354     <section class="book-section" id="book" aria-label="Book a call">
355         <div class="container">
356             <div class="book-wrap reveal">
357                 <p class="book-eyebrow">Let's talk</p>
358                 <h2 class="book-title">30 minutes.<br>No slides.</h2>
359                 <p class="book-sub">Tell me about your content goals. I'll tell you if I can help.</p>
360                 <div class="book-cta-row">
361                     <!-- UPDATE: replace href with your booking link -->
362                     <a href="https://cal.com/salman-adnan/meeting" target="_blank" rel="noopener" class="
363                         btn btn-primary" style="padding:16px 36px;font-size:17px;">
364                         Book a call
365                         <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
366                         stroke-width="2.5" aria-hidden="true">
367                             <path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l-7.5
368                             7.5M21 12H3"/>
369                             </svg>
370                         </a>
371                     </div>
372                     <p class="book-detail">
373                         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" aria-
374                         hidden="true">
375                             <path stroke-linecap="round" stroke-linejoin="round" d="M12 6v6h4.5m4.5 0a9 9 0
376                             11-18 0 9 9 0 0118 0z"/>
377                             </svg>
378                         Free · <span>30 minutes · Zoom</span>
379                     </p>
380                 </div>
381             </div>
382         </div>
383     </section>

```

Listing 15: index.html, lines 348–377

This is the destination of every “Book a call” button, via `id="book"` on line 354. It is the point of the whole document.

Line 356 combines two classes, `book-wrap` for the panel’s appearance and `reveal` so it fades in, the only place in the file where a layout class and a behaviour class share an element.

Line 362 is the booking link, and three of its attributes matter. `target="_blank"` opens it in a new tab so the visitor does not lose the page. `rel="noopener"` is a security measure that must accompany it: without it the newly opened page receives a reference back to this one and can navigate it somewhere else, a trick known as tab-nabbing. Its presence is correct and is the kind of detail usually forgotten.

This booking link is real, and it outlived the design

`cal.com/salman-adnan/meeting` is a genuine personal calendar, set deliberately in commit 014406e. The sibling repository’s equivalent points at `cal.com/davonex/30min`, another project’s calendar, and is a known outstanding placeholder.

More than that, this link is the one part of this design that is demonstrably still in service: fetching the live `salmanadnan.com` on 2026-08-11 found the same link on the page that replaced this one.

The comment above the link is stale

Line 352 still tells the reader to search for `YOUR_CAL_LINK`, and line 361 repeats the instruction. That literal placeholder was replaced on 2026-07-28 and appears nowhere in the file. `ASSETS.md` line 28 repeats the same stale instruction a third time. Somebody following the written instructions would search, find nothing, and have no way to tell whether the job was

done or the instruction was wrong.

Lines 369 to 374 are the reassurance line under the button: a clock icon, then “Free”, then the duration and platform. The middle dots are the U+00B7 character discussed in section 1.4 and render exactly.

Lines 378 to 395: the footer, and the script tag

```

378
379
380 <!-- =====
381      FOOTER
382      ===== -->
383 <footer>
384   <div class="container">
385     <p>
386       <a href="mailto:salman@digitalise.agency">salman@digitalise.agency</a>
387       &nbsp;·&nbsp;
388       <a href="https://digitalise.agency" target="_blank" rel="noopener">digitalise.agency</a>
389     </p>
390   </div>
391 </footer>
392
393 <script src="/js/main.js"></script>
394 </body>
395 </html>

```

Listing 16: index.html, lines 378–395

The footer is one line: an email address and a link to the agency, separated by a middle dot padded with ` `; on each side.

Jargon: Non-breaking space

` ` is a space a browser will not use as a place to wrap a line, and which is not collapsed when several sit together. Here it does the second job: ordinary spaces around the dot would collapse to one, and these keep the separator visually padded.

Line 386 is a `mailto:` link, which opens the visitor’s email program with the address filled in. Line 388 links to the agency site, correctly carrying both `target="_blank"` and `rel="noopener"`.

This footer is better than the sibling’s, which links to `digitalise.agency` as text but points the link at `/`, its own home page. Here the text and the destination agree.

Line 393 loads the JavaScript, and **where it sits is the whole point**. It is the last thing before `</body>`, so by the time the browser reaches it every element already exists. `main.js` begins searching for elements immediately, with no wrapper telling it to wait, and that only works because of this placement. Move this line into the `<head>` and the script would run against an empty document, find nothing, and the page would quietly lose all three behaviours with no error message.

Lines 394 and 395 close the body and the document.

What it connects to

This file names: `new-design-reference/css/site.css` (line 16), `new-design-reference/js/main.js` (line 393), Google Fonts (lines 13 to 15), `cal.com` (line 362), and the agency site (line 388).

This file is named by: nothing inside the repository. The workflow copies the whole folder rather than referring to files individually, and `netlify.toml` publishes the folder as a whole. `ASSETS.md` discusses it at length and cannot affect it.

Internal links: the navigation and hero both point at `#book` (lines 26 and 105), and the hero also points at `#work` (line 111). Those two ids, on lines 354 and 138, are the only link targets in the file.

How it breaks

- **Rename a class here without renaming it in the stylesheet** and that element loses its appearance silently. Nothing warns you. Section 7.2 documents a live instance of the reverse: a stylesheet class that matches nothing here.
- **Move the `<script>` tag into the `<head>`** and all three behaviours stop, with no error. The metrics band would then permanently show four zeros.
- **Insert an element before the placeholder on line 43** and all three floating badges jump, because the stylesheet positions them by child number.
- **Open the file directly off a disk** and it renders unstyled, because of the leading slash on line 16. This looks like catastrophic breakage and is not.
- **Disable JavaScript** and the page is fully readable but static. The metrics band shows four zeros; the three hero statistics still show their real values, which is what saves it.
- **Delete the id on line 354** and every “Book a call” button silently does nothing, because `main.js` returns early when it cannot find a link’s target.

Honest findings for this file

The visible photo placeholder on lines 43 to 46; the missing `og:image` and favicon; seven stale comments (lines 33, 42, 135, 256, 284, 352, 361); a non-clickable logo on line 25; `<em>` used as a colour hook on line 99; the same three numbers stated twice in two formats; play buttons that play nothing; colour literals used where tokens exist (lines 51, 63, 75); and a `case-card-reverse` class that no stylesheet rule matches.

7.2 new-design-reference/css/site.css

CSS, 575 lines. Every visual decision: colours, typefaces, spacing, layout, movement, and how all of it rearranges on a narrow screen. It is the longest file here and, in the judgement of this document, the most valuable thing in the repository.

Where this came from: new-design-reference/css/site.css

Hand-written, and it states its own lineage in its second line: **Personal brand site. Shares design tokens with ai.digitalise.agency.** That is a direct statement by the author that the naming scheme came from the sibling project, and comparing the two token blocks confirms it: identical names, identical structure, different values.

It arrived in the first commit `1f7ed12` and was substantially edited once, by `05210af` (“Differentiate to orange brand”). No generator banner, no vendor prefix block of the sort a preprocessor emits, and no minified section: it is written to be read. **Confidence: certain for hand-written, certain for the token lineage, which rests on the author’s own comment and is corroborated by the files.**

Why it exists

Without this file the page contains every word it needs and looks like a 1994 document: black serif text on white, one column, no colour. Nothing would be broken and nothing would be persuasive. The two-column hero would collapse into a stack, and the three floating badges would appear as three small blocks of text under the placeholder.

Line by line

Lines 1 to 25: the header, and the token block

```

1  /* salmanadnan.com · site.css
2     Personal brand site. Shares design tokens with ai.digitalise.agency. */
3
4  /* --- TOKENS --- */
5  :root {
6     --s0:      #05070d;
7     --s1:      #0e1118;
8     --s2:      #161b26;
9     --ink1:    #f4f7fc;
10    --ink2:    #c3ccdd;
11    --ink3:    #93a0b8;
12    --accent:   #ff6719; /* orange, personal brand, distinct from agency blue */
13    --accentH:  #ff8c4a;
14    --accent2:  #1967ff;
15    --green:    #34d399;
16    --border:   #24201e;
17    --borderS:  #1c1814;
18    --font-d:   'Space Grotesk', ui-sans-serif, system-ui, sans-serif;
19    --font-b:   'Inter', ui-sans-serif, system-ui, sans-serif;
20    --r:        16px;
21    --rsm:      10px;
22    --ease:     cubic-bezier(.22,1,.36,1);
23    --maxw:     1240px;
24    --shadow:   0 8px 40px rgba(0,0,0,.45);
25  }
```

Listing 17: css/site.css, lines 1–25

Lines 1 and 2 are a comment. In CSS, anything between `/*` and `*/` is ignored, and unlike HTML there is only this one comment form. These two lines carry the file’s entire provenance.

Line 4 is the first of twelve section banners drawn with box-drawing characters, the author’s navigation aid.

Line 5 opens the token block. `:root` matches the document’s outermost element, `<html>`. Custom properties defined there are inherited by every element on the page, which is exactly why token blocks are written against `:root`: it is the one place a definition reaches everything.

The nineteen tokens fall into six groups:

Tokens	Line	What the group is for
-s0 to -s2	6–8	Surfaces, darkest to lightest. <code>s0</code> is the page, <code>s1</code> is a card, <code>s2</code> would be a detail inside a card
-ink1 to -ink3	9–11	Text, brightest to dimmest: a heading, body text, a caption
-accent, -accentH, -accent2	12–14	The orange, its hover variant, and the agency blue kept as a secondary
-green	15	The one success colour
-border, -borderS	16–17	A visible hairline, and a subtler one
-font-d, -font-b	18–19	Display typeface and body typeface
-r, -rsm, -ease, -maxw, -shadow	20–24	Corner radius, small radius, motion curve, content width, drop shadow

Table 3: The token vocabulary, identical in naming to the sibling repository and different in almost every value.

Why a numbered scale beats named colours

Calling a colour `-s1` instead of `-dark-navy` names what it is *for* (the surface one step up from the page) rather than how it looks. This pair of repositories proves the point better than any argument could: the two files share every token *name* and differ in nearly every token *value*, so the same stylesheet vocabulary produced a blue agency site and an orange personal one with no renaming at all.

Line 12 carries the rebrand, and it carries its own explanation in an inline comment: `orange`, `personal brand`, distinct from `agency blue`. Line 14 then keeps the agency blue as `-accent2`.

Four tokens are defined and never read

`-s2` (line 8), `-accent2` (line 14), `-rsm` (line 21) and `-shadow` (line 24) are read zero times in this file. Counted mechanically: `grep -c 'var(-accent2)'` returns 0, and likewise for the other three.

`-accent2` is the most interesting of the four. It holds the agency blue, deliberately kept as a secondary colour, and is then never used as one. Meanwhile five other rules in this file hard-code that same blue as a literal rather than reading this token. The colour is used and the token for it is not, which is the worst of both worlds and is the subject of section 7.2's largest finding, below.

Line 22 defines the motion curve. `cubic-bezier(.22,1,.36,1)` describes how an animation's speed varies across its run, by giving two control points of a curve. This one starts fast and settles gently, the shape usually described as “ease out”. Every transition in the file refers to it, so all movement shares one personality.

Line 23 sets the content width to 1240 pixels, 80 narrower than the sibling's 1320, which is the kind of small deliberate difference that shows these files were tuned separately rather than copied wholesale.

Lines 26 to 43: the reset, and the one utility class

```

26
27 *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
28 html { color-scheme: dark; scroll-behavior: smooth; }
29 body {
30     font-family: var(--font-b);
31     background: var(--s0);
32     color: var(--ink1);
33     -webkit-font-smoothing: antialiased;
34     text-rendering: optimizeLegibility;
35     overflow-x: hidden;
36     line-height: 1.5;
37 }
38 img, video, svg { max-width: 100%; display: block; }
39 a { color: inherit; text-decoration: none; }
40 button { cursor: pointer; border: none; font-family: inherit; }
41
42 /* --- UTILITY --- */
43 .container { max-width: var(--maxw); margin: 0 auto; padding: 0 28px; }
```

Listing 18: `css/site.css`, lines 26–43

Line 27 is a reset doing three jobs. The selector `*` means every element; `::before` and `::after` extend that to the two invisible pseudo-elements CSS lets you attach to anything.

`box-sizing: border-box` is the most consequential declaration in the file. By default, an element given a width of 200 pixels and 20 pixels of padding occupies 240: the padding is added outside the stated width. That default surprises everyone. `border-box` makes the stated width final, with padding eating into it. Every layout calculation here assumes it.

`margin: 0; padding: 0` removes the browser's built-in spacing on headings and paragraphs, so all spacing in the design is deliberate rather than inherited from defaults that vary between browsers.

Line 28 does two things to the document root. `color-scheme: dark` tells the browser this page is dark, so scrollbars, selection highlights and form controls are drawn in their dark variants rather than staying stubbornly white. `scroll-behavior: smooth` makes every jump-link glide rather than snap.

Smooth scrolling is implemented twice

Line 28 already makes anchor links glide. `main.js` lines 55 to 62 then intercept every anchor link and call `scrollIntoView({behavior: 'smooth'})` to do the same again. Either alone suffices. The JavaScript copy is the redundant one and it also carries a latent bug, described in section 7.3.

Lines 29 to 37 set the page defaults: body typeface, darkest surface as background, brightest ink as text, and a line height of 1.5, meaning each line occupies one and a half times the font size. That last number is the most important typographic decision in the file and the one most often set badly.

`-webkit-font-smoothing: antialiased` on line 33 thins text rendering on Mac displays. It is a Safari and Chrome specific property with no standard equivalent, and it is the conventional companion to light text on a dark background, which otherwise looks heavier than intended.

`overflow-x: hidden` on line 35 forbids horizontal scrolling, and here it is load-bearing rather than cosmetic: the hero is a two-column grid whose left column is a full-height photo frame, and the floating badges are positioned by percentage inside it.

Line 38 makes images, videos and SVGs never exceed their container and makes them block-level. That second part fixes a genuinely confusing default: images are inline by default, which reserves space beneath them for the descenders of text that is not there, producing a mysterious few pixels of gap under every image.

Line 39 strips links of their blue colour and underline, making them inherit the surrounding colour. That is a design choice and an accessibility trade: colour alone now distinguishes a link from text, which fails the usual guidance. Every link here is either a button or a footer item, so the practical harm is small.

Line 40 styles buttons, and is dead in the strict sense that this page contains no `<button>` element at all. It costs nothing and would come alive the moment one was added.

Line 43 is the only utility class: a centred column no wider than 1240 pixels with 28 pixels of breathing room each side. `margin: 0 auto` is the standard centring idiom, meaning no vertical margin and equal automatic margins left and right. Every section below the hero wraps its content in this; the hero deliberately does not, because it spans the full window.

Lines 44 to 72: the button system, and the first missed rebrand¹

```

44
45 .btn {
46   display: inline-flex;
47   align-items: center;
48   gap: 8px;
49   padding: 13px 28px;
50   border-radius: 100px;
51   font-family: var(--font-d);
52   font-size: 15px;
53   font-weight: 600;
54   letter-spacing: -0.01em;
55   transition: transform 0.18s var(--ease), box-shadow 0.18s var(--ease),
56               background 0.18s var(--ease);
57 }
58 .btn-primary {

```

```

59   background: var(--accent);
60   color: #fff;
61 }
62 .btn-primary:hover {
63   background: var(--accentH);
64   transform: translateY(-2px);
65   box-shadow: 0 8px 24px rgba(25,103,255,.4);
66 }
67 .btn-ghost {
68   background: transparent;
69   color: var(--ink2);
70   border: 1px solid var(--border);
71 }
72 .btn-ghost:hover { background: var(--s1); color: var(--ink1); }

```

Listing 19: `css/site.css`, lines 44–72

This is the two-class pattern the HTML uses: `.btn` holds everything both buttons share and `.btn-primary` or `.btn-ghost` adds only what differs. Add a third style and you write four lines, not thirty.

`display: inline-flex` on line 46 makes the button a flex container sitting in the flow of text rather than on its own line. That matters because the primary buttons contain both a word and an icon, and flex is what lines those up.

Jargon: Flexbox

A layout system arranging an element’s children in a row or a column and controlling how they align and share leftover space. Set `display: flex` on a parent and its children become flex items. `align-items: center` lines them up on their centres, which is how the icon in a button sits level with the text rather than on the baseline.

`gap: 8px` on line 48 puts eight pixels between word and icon without either needing a margin. Before `gap` existed this required fiddly rules targeting all-but-the-first child, and its arrival is one of the genuine improvements in modern CSS.

`border-radius: 100px` on line 50 makes a pill. Any radius larger than half the element’s height gives a perfect semicircular end, so 100 is simply comfortably larger than needed.

Lines 55 and 56 declare the transition: three named properties, each over 0.18 seconds, each using the shared curve. Listing properties explicitly rather than writing `transition: all` is better practice, because `all` makes the browser watch every property and can animate things you never intended.

An orange button that casts a blue shadow

Line 65 sets the hover shadow to `rgba(25,103,255,.4)`. That is `#1967ff`, the agency blue, at 40% opacity. The button itself is orange (`-accent`, line 12).

Orange and blue sit close to opposite each other on the colour wheel, so this does not read as a subtle mismatch; it reads as a mistake. It is the most visible of the five surviving blue literals catalogued below, and it is a one-line fix: the value should be `rgba(255,103,25,.4)`, or better still derived from the token.

The cause is legible from the history. Commit 05210af changed the tokens to orange; this line never used a token, so the change could not reach it.

Jargon: rgba and opacity

`rgba(25,103,255,.4)` names a colour by red, green and blue amounts from 0 to 255, plus an opacity from 0 (invisible) to 1 (solid). It is the same blue as `#1967ff`, expressed in the form that allows transparency.

Neither button has a focus state

Neither `.btn-primary` nor `.btn-ghost` defines `:focus` or `:focus-visible`. A visitor navigating by keyboard rather than mouse gets whatever outline the browser draws by default, which on a dark background is often nearly invisible. Since these are the page's only interactive elements, this is the most consequential accessibility gap in the stylesheet, and it is three lines to fix.

Lines 73 to 88: the two reveal mechanisms

```

73
74 /* --- REVEAL --- */
75 .reveal {
76   opacity: 0;
77   transform: translateY(28px);
78   transition: opacity 0.7s var(--ease), transform 0.7s var(--ease);
79 }
80 .reveal.visible { opacity: 1; transform: none; }
81
82 .stagger > * {
83   opacity: 0;
84   transform: translateY(20px);
85   transition: opacity 0.6s var(--ease), transform 0.6s var(--ease);
86   transition-delay: calc(var(--i, 0) * 90ms);
87 }
88 .stagger.visible > * { opacity: 1; transform: none; }

```

Listing 20: `css/site.css`, lines 73–88

These sixteen lines are the entire animation contract between the stylesheet and the JavaScript: the CSS defines both states and the transition between them, and the JavaScript's only job is to add the class `visible` at the right moment.

`.reveal` starts invisible and 28 pixels low. `.reveal.visible`, which matches an element carrying *both* classes, returns it to fully visible and its natural position. Because a transition is declared, the browser animates between the two over 0.7 seconds rather than snapping.

The stagger is the same idea with one addition. `.stagger > *` means every direct child of a staggered container, and line 86 delays each child by `calc(var(--i, 0) * 90ms)`: the number the HTML wrote inline, multiplied by 90 milliseconds. Child 0 starts immediately, child 5 starts 450 milliseconds later.

Jargon: `calc`, and a custom property's fallback

`calc()` performs arithmetic in CSS and can mix units. `var(--i, 0)` reads the custom property `-i` and falls back to 0 if it was never set, which keeps this rule safe on any element whose inline `-i` was forgotten: it gets no delay instead of breaking.

Why the animation lives in CSS and the trigger lives in JavaScript

This split is why the page degrades well, and it is also its one real hazard. If JavaScript never runs, the class `visible` is never added and every revealed element stays at opacity zero, invisible forever. That would be catastrophic, and it is exactly why line 573 exists: the reduced-motion block at the end of the file forces `.reveal` and `.stagger > *` back to full opacity. That block is doing double duty as an accessibility feature and as a safety net.

Lines 89 to 122: the navigation bar

```

89
90 /* --- NAV --- */
91 .nav {
92   position: fixed;

```

```

93   top: 0; left: 0; right: 0;
94   z-index: 100;
95   padding: 18px 28px;
96   display: flex;
97   align-items: center;
98   justify-content: space-between;
99   transition: background 0.3s, border-color 0.3s, box-shadow 0.3s;
100 }
101 .nav.scrolled {
102   background: rgba(5,7,13,0.88);
103   border-bottom: 1px solid var(--border);
104   box-shadow: 0 4px 24px rgba(0,0,0,.3);
105   backdrop-filter: blur(16px);
106   -webkit-backdrop-filter: blur(16px);
107 }
108 .nav-inner {
109   max-width: var(--maxw);
110   width: 100%;
111   margin: 0 auto;
112   display: flex;
113   align-items: center;
114   justify-content: space-between;
115 }
116 .nav-logo {
117   font-family: var(--font-d);
118   font-size: 17px;
119   font-weight: 700;
120   letter-spacing: -0.02em;
121   color: var(--ink1);
122 }

```

Listing 21: `css/site.css`, lines 89–122

`position: fixed` on line 92 pins the bar to the window rather than the document, so it stays put while the page scrolls beneath it. Line 93 pins it to the top and stretches it across.

`z-index: 100` on line 94 controls stacking order: when elements overlap, the higher number is drawn in front. 100 is arbitrary and generous, chosen so nothing else can accidentally cover the bar.

Lines 101 to 107 are the scrolled state, the class `main.js` adds once the page has moved more than 40 pixels: a background at 88% opacity, a bottom hairline, and a shadow.

`backdrop-filter: blur(16px)` on line 105 is the interesting one: it blurs whatever is *behind* the element, producing the frosted-glass effect that only works because the background is slightly transparent. Line 106 repeats it with a `-webkit-` prefix for Safari, which needed the prefixed form for years.

A doubled flex layout

`.nav` sets `display: flex` with `justify-content: space-between` (lines 96 to 98), and `.nav-inner` sets the same three properties again (lines 112 to 114). The outer one has a single child, so its `space-between` has nothing to space. It is harmless and redundant, and it makes the layout harder to reason about than it needs to be. The sibling repository has exactly the same doubling, so it came from the shared skeleton.

Lines 116 to 122 style the logo. Note there is no rule for a logo mark or an accent-coloured fragment, unlike the sibling: this logo is one span of plain text, matching the simpler markup described in section 7.1.

Lines 123 to 133: the hero as a two-column grid

```

123
124 /* --- HERO --- */

```

```
125 .hero {  
126   min-height: 100vh;  
127   display: grid;  
128   grid-template-columns: 1fr 1fr;  
129   align-items: center;  
130   gap: 0;  
131   padding-top: 80px;  
132   overflow: hidden;  
133 }
```

Listing 22: `css/site.css`, lines 123–133

This is where the two siblings genuinely diverge. The other repository’s hero is a single centred column over a field of drifting cards; this one is a two-column grid, photograph on the left and words on the right.

Jargon: CSS Grid

A layout system for two dimensions, as opposed to flexbox’s one. `grid-template-columns: 1fr 1fr` means two columns each taking one equal share of the space. `1fr` is a fraction of what remains after any fixed sizes are accounted for.

`min-height: 100vh` on line 126 makes the hero at least as tall as the window. `vh` means one percent of the viewport height. `padding-top: 80px` on line 131 leaves room for the fixed navigation bar, which is removed from the normal flow and would otherwise sit on top of the content.

Lines 134 to 148: the photo column

```
134  
135 /* photo side */  
136 .hero-photo-side {  
137   position: relative;  
138   height: 100vh;  
139   display: flex;  
140   align-items: center;  
141   justify-content: center;  
142   overflow: hidden;  
143 }  
144 .hero-photo-frame {  
145   position: relative;  
146   width: 100%;  
147   height: 100%;  
148 }
```

Listing 23: `css/site.css`, lines 134–148

`position: relative` on line 137 looks like it does nothing and is essential. It establishes this element as the reference point for any absolutely positioned descendant, which here means the three floating badges. Without it they would position themselves against the whole window instead of against the photo column.

Jargon: Positioning: relative and absolute

`position: absolute` takes an element out of the normal flow and places it at coordinates you give, measured from the nearest ancestor that is itself positioned, which in practice means the nearest one with `position: relative`. This pairing, a relative parent and absolute children, is the standard way to place things inside a box.

Line 138 sets the column’s height to `100vh`, the full window height, which is what makes the photograph bleed to the top and bottom edges of the screen.

Lines 149 to 179: the placeholder, styled with real care

```

149
150 /* Photo placeholder - replace with  */
151 .hero-photo-placeholder {
152     width: 100%;
153     height: 100%;
154     background: linear-gradient(160deg, #1a0800 0%, #4a1800 40%, #05070d 100%);
155     display: flex;
156     align-items: center;
157     justify-content: center;
158     flex-direction: column;
159     gap: 16px;
160     color: rgba(255,255,255,.15);
161     font-family: var(--font-d);
162 }
163 .hero-photo-placeholder .placeholder-initial {
164     font-size: 120px;
165     font-weight: 700;
166     letter-spacing: -0.05em;
167     line-height: 1;
168     background: linear-gradient(135deg, rgba(255,103,25,.6), rgba(255,140,74,.3));
169     -webkit-background-clip: text;
170     -webkit-text-fill-color: transparent;
171     background-clip: text;
172 }
173 .hero-photo-placeholder .placeholder-note {
174     font-size: 13px;
175     color: rgba(255,255,255,.25);
176     text-align: center;
177     max-width: 220px;
178     line-height: 1.5;
179 }

```

Listing 24: css/site.css, lines 149–179

These thirty lines style the placeholder that section 7.1 identified as the most consequential thing in the repository, and they are worth reading for what they say about intent.

Line 154 fills the frame with a three-stop gradient running from a near-black brown, through a deep orange, to the page background. That is a considered stand-in for a photograph: it establishes the brand colour and fades into the page rather than ending at a hard edge.

Lines 163 to 172 style the giant letter, and they use a genuinely advanced technique.

How the letter is filled with a gradient

You cannot pour a gradient into text directly. The trick on lines 168 to 171 is three steps: give the element a gradient background, tell the browser to clip that background to the shape of the text (`background-clip: text`), and then make the text itself transparent (`-webkit-text-fill-color: transparent`) so the clipped background shows through the letterforms.

Line 171 sets the standard `background-clip: text` and line 169 the prefixed `-webkit-` form, which is still needed for full support. Note the third line has no unprefixed equivalent at all: `-webkit-text-fill-color` is a non-standard property that every major browser implements, and this effect depends on it.

Lines 173 to 179 style the instruction text: 13 pixels, white at 25% opacity, centred, capped at 220 pixels wide.

The placeholder note is deliberately styled to be readable

This is the detail that settles how to describe the placeholder. The note is not raw unstyled debris: it has been given a size, a colour, an alignment, a maximum width and a line height. Somebody laid it out to be read.

Read charitably, that is a designer making an unfinished state look tidy in a review. Read plainly, it means a visitor to the published page would see a legible instruction addressed to a developer. Both readings are consistent with the evidence and the second is what a visitor experiences, which is why section 14 treats it as the project's defining unfinished element.

Lines 180 to 200: the two edge fader

```

180
181 /* fade gradient on the right edge so text side blends */
182 .hero-photo-frame::after {
183     content: '';
184     position: absolute;
185     top: 0; right: 0;
186     width: 40%;
187     height: 100%;
188     background: linear-gradient(to right, transparent, var(--s0));
189     pointer-events: none;
190 }
191 /* fade at bottom */
192 .hero-photo-frame::before {
193     content: '';
194     position: absolute;
195     bottom: 0; left: 0; right: 0;
196     height: 30%;
197     background: linear-gradient(to top, var(--s0), transparent);
198     pointer-events: none;
199     z-index: 1;
200 }
```

Listing 25: `css/site.css`, lines 180–200

Two pseudo-elements, and they solve the problem a hard-edged half-page image creates.

Jargon: Pseudo-element

An extra box CSS creates inside an element with no corresponding tag in the HTML, written `::before` or `::after`. It needs the `content` property to exist at all, which is why lines 183 and 193 set `content: ''`: an empty string is enough to bring it into being. It is the standard way to draw a decorative shape without polluting the markup.

The `::after` (lines 182 to 190) covers the rightmost 40% of the photo column with a gradient running from transparent to the page background, so the photograph dissolves into the text column instead of ending at a visible seam. The `::before` (lines 192 to 200) does the same across the bottom 30%.

Both carry `pointer-events: none`, which makes them transparent to the mouse so clicks pass through. Without it these two invisible layers would sit over the photo column and swallow any interaction beneath them.

Why the bottom fade needs a z-index and the right fade does not

Line 199 gives the `::before` a `z-index: 1`. The reason is ordering: without an explicit stacking position, a `::before` is painted *behind* its element's content, while an `::after` is painted in front. So the right fade works by default and the bottom fade would be hidden behind the placeholder until lifted.

The badges then sit at `z-index: 2` (line 214), one layer above the bottom fade, which is what keeps them crisp while the photograph dims underneath. Three elements, three deliberate layers.

Lines 201 to 241: the floating badges

```

201
202 /* floating badges on the photo */
203 .hero-float-badge {
204     position: absolute;
205     background: rgba(13,18,32,0.85);
206     backdrop-filter: blur(12px);
207     -webkit-backdrop-filter: blur(12px);
208     border: 1px solid var(--border);
209     border-radius: 12px;
210     padding: 12px 16px;
211     display: flex;
212     align-items: center;
213     gap: 10px;
214     z-index: 2;
215     animation: badgeFloat 6s ease-in-out infinite;
216 }
217 .hero-float-badge:nth-child(2) { bottom: 25%; left: 8%; animation-delay: 0s; animation-
    duration: 7s; }
218 .hero-float-badge:nth-child(3) { bottom: 40%; right: 12%; animation-delay: 2s; animation-
    duration: 6s; }
219 .hero-float-badge:nth-child(4) { top: 22%; left: 10%; animation-delay: 1s; animation-
    duration: 8s; }
220
221 @keyframes badgeFloat {
222     0%, 100% { transform: translateY(0); }
223     50%      { transform: translateY(-8px); }
224 }
225 .badge-icon {
226     width: 36px; height: 36px;
227     border-radius: 10px;
228     display: flex; align-items: center; justify-content: center;
229 }
230 .badge-icon-blue { background: rgba(25,103,255,.15); }
231 .badge-icon-orange { background: rgba(255,103,25,.15); }
232 .badge-icon-green { background: rgba(52,211,153,.12); }
233 .badge-icon svg { width: 18px; height: 18px; }
234 .badge-text-main {
235     font-family: var(--font-d);
236     font-size: 14px;
237     font-weight: 700;
238     color: var(--ink1);
239     white-space: nowrap;
240 }
241 .badge-text-sub { font-size: 11px; color: var(--ink3); white-space: nowrap; }

```

Listing 26: `css/site.css`, lines 201–241

Lines 203 to 216 define a badge: absolutely positioned, dark and semi-transparent with a 12-pixel blur behind it, bordered, rounded, laid out as a row with a gap, lifted to `z-index: 2`, and animated forever.

Note line 205: `rgba(13,18,32,0.85)`.

A colour left behind by the rebrand, in a subtler way

`rgb(13,18,32)` is `#0d1220`, which is the *sibling repository*'s `-s1` surface colour. This file's own `-s1` is `#0e1118`, a slightly different, less blue dark. So the badge background is the other project's card colour, written as a literal, at 85% opacity.

Nobody would notice: the two values differ by a few points of blue and the badge sits over a gradient at 85% opacity. It is included here because it is evidence about how the rebrand was done, not because it is a visible defect. The tokens were updated and every literal in the file was left exactly as inherited.

Lines 217 to 219 place the three badges individually, and this is where the HTML's child order becomes load-bearing. `:nth-child(2)` matches the second child of its parent, so these three rules assume the placeholder occupies child 1, as section 7.1 explained. Each badge gets a different corner, a different animation delay and a different duration (7, 6 and 8 seconds), which is what stops the three drifting in visible unison.

Lines 221 to 224 define the drift itself: a gentle rise of 8 pixels at the halfway point and back. Grouping `0%, 100%` on one line is the idiom for an animation returning to where it started.

Jargon: Keyframes

An animation defined by describing the element at particular points through its run, given as percentages of the total duration. The browser fills in every moment between. `0%` is the start and `100%` the end.

Lines 230 to 232 are the three tinted icon chips: blue, orange and green, each the matching colour at 12 to 15% opacity. As section 7.1 noted, these three are the one place blue is plainly intentional.

Line 239 and line 241 both set `white-space: nowrap`, stopping “1.6M+ views” and “organic, for clients” wrapping onto two lines and making the badge grow taller than its neighbours.

Lines 242 to 301: the content column

```

242
243 /* content side */
244 .hero-content-side {
245     padding: 0 48px 0 20px;
246     display: flex;
247     flex-direction: column;
248     justify-content: center;
249     gap: 32px;
250 }
251 .hero-tag {
252     display: inline-flex;
253     align-items: center;
254     gap: 8px;
255     background: rgba(25,103,255,.1);
256     border: 1px solid rgba(25,103,255,.25);
257     border-radius: 100px;
258     padding: 6px 14px;
259     font-size: 13px;
260     font-weight: 600;
261     color: var(--accentH);
262     width: fit-content;
263 }
264 .hero-h1 {
265     font-family: var(--font-d);
266     font-size: clamp(40px, 5vw, 70px);
267     font-weight: 700;
268     letter-spacing: -0.03em;
269     line-height: 1.04;

```

```

270     color: var(--ink1);
271 }
272 .hero-h1 em { font-style: normal; color: var(--accent); }
273 .hero-descriptor {
274     font-family: var(--font-d);
275     font-size: 20px;
276     font-weight: 600;
277     color: var(--ink3);
278     letter-spacing: -0.01em;
279 }
280 .hero-actions {
281     display: flex;
282     align-items: center;
283     gap: 12px;
284     flex-wrap: wrap;
285 }
286 .hero-mini-stats {
287     display: flex;
288     align-items: center;
289     gap: 28px;
290     padding-top: 4px;
291     border-top: 1px solid var(--borderS);
292 }
293 .hero-mini-stat { text-align: left; }
294 .hero-mini-num {
295     font-family: var(--font-d);
296     font-size: 24px;
297     font-weight: 700;
298     color: var(--ink1);
299     letter-spacing: -0.03em;
300 }
301 .hero-mini-lbl { font-size: 12px; color: var(--ink3); }

```

Listing 27: css/site.css, lines 242–301

Line 244 lays the right column out as a vertical flex column with a 32-pixel gap, so its five children space themselves without margins. The asymmetric padding (0 48px 0 20px) pulls the text slightly left, toward the fading edge of the photograph.

Lines 251 to 263 are the small pill above the name, and it contains the second missed rebrand.

A blue pill containing orange text

Lines 255 and 256 tint the pill with `rgba(25,103,255,...)`, the agency blue, while line 261 colours its text `var(-accentH)`, which is the orange `#ff8c4a`.

So the element reads its foreground from a token and its background from a literal, and after the rebrand the two no longer agree. On screen it is a blue-tinted capsule with orange text inside it, which looks like neither brand.

`width: fit-content` on line 262 is worth noting separately and is correct: in a flex column, a child stretches to full width by default, so without this the pill would span the entire column instead of hugging its text.

Line 266 is the most useful single line in the file to learn from.

Jargon: clamp

`clamp(minimum, preferred, maximum)` takes the preferred value but never lets it fall below the minimum or rise above the maximum. Here the preferred value is `5vw`, 5% of the window width, so the headline scales continuously between a floor of 40 pixels and a ceiling of 70. One line replaces what used to need several media queries.

Line 268 sets `letter-spacing: -0.03em`, pulling letters closer together. Large display type looks loose at its default spacing, and negative tracking at large sizes is standard practice. The unit `em` means “relative to this element’s font size”, so the tightening scales with the text.

Line 272 is the rule flagged in section 7.1: it takes the `<em>` tag, removes the italics it normally implies, and colours it with the accent instead.

Lines 286 to 301 build the three small statistics: a flex row with a 28-pixel gap above a hairline, each statistic a 24-pixel number over a 12-pixel label.

Lines 302 to 321: the work section, and the shared headings

```

302
303 /* --- WORK GRID --- */
304 .work-section { padding: 120px 0 80px; }
305 .section-kicker {
306     font-size: 12px;
307     font-weight: 700;
308     text-transform: uppercase;
309     letter-spacing: 0.1em;
310     color: var(--accent);
311     margin-bottom: 14px;
312 }
313 .section-title {
314     font-family: var(--font-d);
315     font-size: clamp(28px, 4vw, 48px);
316     font-weight: 700;
317     letter-spacing: -0.025em;
318     line-height: 1.1;
319     color: var(--ink1);
320     margin-bottom: 52px;
321 }
```

Listing 28: `css/site.css`, lines 302–321

Although they sit under the work banner, `.section-kicker` and `.section-title` are used by three sections. Their placement here is a small organisational untidiness: a reader looking for the heading styles would reasonably expect them near the top with the other shared pieces.

The kicker is the small uppercase line (“The work”, “Case studies”) and the title is the large heading beneath it. Note the tracking is positive on the kicker and negative on the title. That is not an inconsistency but a rule of typography: large text needs tightening and small uppercase text needs opening up or it becomes a smudge.

The kicker’s contrast is worth measuring

`--accent` is now `#ff6719`, a saturated orange, set at 12 pixels uppercase on a near-black background. Small text needs more contrast than large text, not less. This document does not have a measured contrast ratio for that exact pairing and so does not assert a failure, but it is the first thing to check in an accessibility audit. Orange on near-black generally fares better than the sibling’s blue on near-black, so this is the less likely of the two to fail.

Lines 322 to 388: the project cards

```

322
323 .work-grid {
324     display: grid;
325     grid-template-columns: repeat(3, 1fr);
326     gap: 14px;
327 }
328 .work-card {
329     background: var(--s1);
330     border: 1px solid var(--border);
331     border-radius: var(--r);
```

```

332     overflow: hidden;
333     transition: transform 0.22s var(--ease), border-color 0.22s, box-shadow 0.22s;
334 }
335 .work-card:hover {
336     transform: translateY(-4px);
337     border-color: rgba(25,103,255,.35);
338     box-shadow: 0 16px 48px rgba(0,0,0,.4);
339 }
340 .work-thumb {
341     aspect-ratio: 16/9;
342     position: relative;
343     display: flex; align-items: center; justify-content: center;
344     overflow: hidden;
345 }
346 .work-thumb-overlay {
347     position: absolute; inset: 0;
348     background: rgba(5,7,13,.2);
349     display: flex; align-items: center; justify-content: center;
350     opacity: 0;
351     transition: opacity 0.22s;
352 }
353 .work-card:hover .work-thumb-overlay { opacity: 1; }
354 .work-play {
355     width: 48px; height: 48px;
356     border-radius: 50%;
357     background: rgba(255,255,255,.15);
358     backdrop-filter: blur(8px);
359     display: flex; align-items: center; justify-content: center;
360     border: 1.5px solid rgba(255,255,255,.3);
361 }
362 .work-play svg { width: 20px; height: 20px; color: #fff; margin-left: 2px; }
363 .work-badge {
364     position: absolute; top: 10px; right: 10px;
365     background: rgba(5,7,13,.75);
366     backdrop-filter: blur(8px);
367     border: 1px solid var(--border);
368     border-radius: 20px;
369     padding: 4px 10px;
370     font-size: 11px; font-weight: 600;
371     color: var(--green);
372 }
373 .wt1 { background: linear-gradient(135deg,#2a1000,#5a2800); }
374 .wt2 { background: linear-gradient(135deg,#2a1800,#5a3000); }
375 .wt3 { background: linear-gradient(135deg,#1a0a00,#3a1800); }
376 .wt4 { background: linear-gradient(135deg,#300800,#602000); }
377 .wt5 { background: linear-gradient(135deg,#200e00,#482800); }
378 .wt6 { background: linear-gradient(135deg,#281400,#503000); }
379 .work-info { padding: 16px; }
380 .work-client {
381     font-size: 11px; color: var(--ink3); font-weight: 600;
382     text-transform: uppercase; letter-spacing: 0.06em; margin-bottom: 4px;
383 }
384 .work-title {
385     font-family: var(--font-d);
386     font-size: 15px; font-weight: 600;
387     color: var(--ink1); letter-spacing: -0.01em;
388 }

```

Listing 29: css/site.css, lines 322–388

`overflow: hidden` on line 332 is what makes the rounded corners work: without it the gradient thumbnail inside would square off the top two corners it is supposed to follow.

`aspect-ratio: 16/9` on line 341 reserves a widescreen-shaped area regardless of width. This is worth appreciating: the layout will not shift when a real image eventually loads, because the

space is already the right shape. It is the modern replacement for a well-known hack involving percentage padding.

The overlay (lines 346 to 353) sits over the whole thumbnail at zero opacity and fades to one on hover, revealing the play button beneath a 20% black wash. The play button (lines 354 to 361) is a frosted circle: white at 15% opacity with an 8-pixel backdrop blur and a brighter rim.

Line 362 gives the play triangle `margin-left: 2px`. That is not an accident. A triangle's visual centre of mass sits left of its geometric centre, so a perfectly centred play icon looks slightly left. Nudging it two pixels right is a standard correction, and noticing it is a good sign that whoever built this cared about detail.

The third missed rebrand

Line 337 sets the hover border to `rgba(25,103,255,.35)`, the agency blue, on a card in an orange-branded site. Same cause, same one-line fix as line 65.

Lines 373 to 378 are the six placeholder gradients, and they *were* rebranded: every one is a two-stop orange or brown blend, where the sibling's equivalents run through blue, teal, violet and green. So the rebrand did reach these six literals and missed five others, which tells you it was done by eye rather than by search.

Lines 380 to 388 style the two lines of text under each thumbnail: a small uppercase client name in the dimmest ink, and the result in the display typeface at 15 pixels. This is the only place in the file where the display typeface does ordinary work at a small size.

Lines 389 to 419: the metrics band

```

389
390 /* --- METRICS --- */
391 .metrics-section {
392     padding: 80px 0;
393     border-top: 1px solid var(--borderS);
394     border-bottom: 1px solid var(--borderS);
395 }
396 .metrics-grid {
397     display: grid;
398     grid-template-columns: repeat(4, 1fr);
399     gap: 2px;
400 }
401 .metric-item {
402     padding: 36px 24px;
403     text-align: center;
404     border-right: 1px solid var(--borderS);
405 }
406 .metric-item:last-child { border-right: none; }
407 .metric-num {
408     font-family: var(--font-d);
409     font-size: clamp(34px, 4vw, 56px);
410     font-weight: 700;
411     letter-spacing: -0.03em;
412     line-height: 1;
413     color: var(--ink1);
414     margin-bottom: 8px;
415 }
416 .metric-label {
417     font-size: 13px; color: var(--ink3); font-weight: 500;
418     text-transform: uppercase; letter-spacing: 0.06em;
419 }
```

Listing 30: `css/site.css`, lines 389–419

Four equal columns with a 2-pixel gap, and the separators drawn as a right border on every item (line 404) with the last one removed (line 406).

Note that this is the only place in the file that draws dividing lines this way. The sibling repository contains a second, cleverer technique for the same job (a grid whose gaps show a coloured layer beneath), used there on its services grid. That technique does not appear here, because this file has no equivalent grid, so the inconsistency the sibling has within itself does not exist in this one.

Lines 407 to 415 style the big numbers: display typeface, scaling between 34 and 56 pixels, with `line-height: 1`. That last is deliberate. The page-wide default of 1.5 is right for paragraphs and wrong for a large number, where it leaves a visible gap above and below.

Lines 420 to 479: the case-study cards, and three dead rules

```

420
421 /* --- CASE STUDIES --- */
422 .cases-section { padding: 100px 0; }
423 .case-card {
424     display: grid;
425     grid-template-columns: 1fr 1fr;
426     gap: 48px;
427     align-items: center;
428     background: var(--s1);
429     border: 1px solid var(--border);
430     border-radius: 24px;
431     padding: 48px;
432     margin-bottom: 16px;
433     transition: border-color 0.22s;
434 }
435 .case-card:hover { border-color: rgba(25,103,255,.3); }
436 .case-card.reverse { direction: rtl; }
437 .case-card.reverse > * { direction: ltr; }
438 .case-visual {
439     border-radius: var(--r);
440     overflow: hidden;
441     aspect-ratio: 16/10;
442     position: relative;
443 }
444 .cv1 { background: linear-gradient(135deg,#2a1000,#5a2800); }
445 .cv2 { background: linear-gradient(135deg,#300800,#602000); }
446 .case-visual-inner {
447     position: absolute; inset: 0;
448     display: flex; align-items: flex-end;
449     padding: 20px;
450     background: linear-gradient(to top, rgba(5,7,13,.8) 0%, transparent 60%);
451 }
452 .case-visual-metrics {
453     display: flex; gap: 12px; flex-wrap: wrap;
454 }
455 .cvm {
456     background: rgba(5,7,13,.7);
457     backdrop-filter: blur(8px);
458     border: 1px solid var(--border);
459     border-radius: 20px;
460     padding: 6px 12px;
461     display: flex; flex-direction: column; gap: 1px;
462 }
463 .cvm-num {
464     font-family: var(--font-d);
465     font-size: 16px; font-weight: 700;
466     color: var(--ink1); letter-spacing: -0.02em;
467 }
468 .cvm-lbl { font-size: 10px; color: var(--ink3); text-transform: uppercase; letter-spacing:
    0.06em; }
469 .case-content { display: flex; flex-direction: column; gap: 20px; }
470 .case-client-name {

```

```

471   font-size: 12px; font-weight: 700;
472   color: var(--accent); text-transform: uppercase; letter-spacing: 0.08em;
473 }
474 .case-title {
475   font-family: var(--font-d);
476   font-size: clamp(20px, 2.5vw, 30px);
477   font-weight: 700; letter-spacing: -0.02em;
478   color: var(--ink1); line-height: 1.2;
479 }

```

Listing 31: `css/site.css`, lines 420–479

Each case card is a two-column grid with a 48-pixel gap: a visual panel and a block of text, vertically centred against each other.

Lines 436 and 437 are the dead rules identified in section 7.1.

Three rules that have never applied, and the mechanism they were replaced by

`.case-card.reverse` means “an element carrying both the class `case-card` and the class `reverse`”. The HTML at line 321 writes `class="case-card case-card-reverse reveal"`. A hyphen does not create a second class, so this selector matches nothing, and neither does its partner on line 437 or the third instance in the responsive block at line 559.

What they were trying to do is elegant and worth understanding even though it is dead. Setting `direction: rtl` on a grid container reverses the order its children are placed in, and setting `direction: ltr` on the children puts their text back the right way round. Two lines, and a two-column layout mirrors itself.

What actually happens instead is that `index.html` sets `order:2` and `order:1` inline on the two children, which achieves the same result by a different route. So the design is correct, three rules are dead, and nothing reported it. This is the clearest illustration in either sibling of what “silent failure” means: an entire mechanism can be broken while the page looks right, because a second mechanism quietly does the same job.

The fourth missed rebrand

Line 435 sets the case card’s hover border to `rgba(25,103,255,.3)`, the agency blue again.

Lines 446 to 451 lay the metric chips over the bottom of each visual, with a gradient from 80% opaque page colour at the bottom to transparent at 60% height, so the chips sit on a dark base and the top of the visual stays clear. Lines 455 to 468 style the chips themselves: frosted, rounded, a number over a tiny uppercase label.

Lines 480 to 524: the booking panel

```

480
481 /* --- BOOK A CALL --- */
482 .book-section { padding: 80px 0 140px; }
483 .book-wrap {
484   background: var(--s1);
485   border: 1px solid var(--border);
486   border-radius: 24px;
487   padding: 72px 48px;
488   text-align: center;
489   position: relative;
490   overflow: hidden;
491 }
492 .book-wrap::before {
493   content: '';
494   position: absolute;
495   top: -100px; left: 50%;
496   transform: translateX(-50%);

```

```

497 width: 600px; height: 300px;
498 background: radial-gradient(ellipse, rgba(25,103,255,.1) 0%, transparent 70%);
499 pointer-events: none;
500 }
501 .book-eyebrow {
502   font-size: 12px; font-weight: 700;
503   text-transform: uppercase; letter-spacing: 0.1em;
504   color: var(--accent); margin-bottom: 16px;
505 }
506 .book-title {
507   font-family: var(--font-d);
508   font-size: clamp(28px, 4vw, 48px);
509   font-weight: 700; letter-spacing: -0.025em;
510   color: var(--ink1); margin-bottom: 12px;
511 }
512 .book-sub {
513   font-size: 16px; color: var(--ink3); margin-bottom: 36px;
514 }
515 .book-cta-row {
516   display: flex; align-items: center; justify-content: center;
517   gap: 14px; flex-wrap: wrap;
518 }
519 .book-detail {
520   display: flex; align-items: center; justify-content: center;
521   gap: 6px; font-size: 13px; color: var(--ink3); margin-top: 20px;
522 }
523 .book-detail svg { width: 15px; height: 15px; }
524 .book-detail span { color: var(--ink2); }

```

Listing 32: css/site.css, lines 480–524

Lines 492 to 500 add a glow above the panel using a `::before`: a 600 by 300 ellipse positioned 100 pixels above the panel’s top edge and centred with the standard `left: 50% plus translateX(-50%)` idiom. Because the panel has `overflow: hidden`, only the lower part of the glow is visible, which makes it read as light spilling in from above rather than as a floating oval.

`pointer-events: none` on line 499 appears for the third time in the file, for the same reason as before: a decorative layer must not intercept the click aimed at the booking button.

The fifth and last missed rebrand

Line 498 makes that glow `rgba(25,103,255,.1)`, the agency blue, on the most important element of the page. It is subtle at 10% opacity, and it means the halo around the call to action is the wrong brand colour.

That completes the catalogue: lines 65, 255, 256, 337, 435 and 498, five rules and six declarations, all inherited from the sibling and all missed by a rebrand that correctly updated the tokens and the six thumbnail gradients.

Lines 525 to 535: the footer

```

525
526 /* --- FOOTER --- */
527 footer {
528   border-top: 1px solid var(--borderS);
529   padding: 32px 28px;
530   text-align: center;
531   font-size: 13px;
532   color: var(--ink3);
533 }
534 footer a { color: var(--ink3); transition: color 0.15s; }
535 footer a:hover { color: var(--ink1); }

```

Listing 33: css/site.css, lines 525–535

Deliberately unremarkable: one hairline, centred dim text, and links that brighten on hover over 0.15 seconds.

Lines 536 to 575: the responsive rules and the accessibility block

```

536
537 /* --- RESPONSIVE --- */
538 @media (max-width: 960px) {
539   .hero {
540     grid-template-columns: 1fr;
541     min-height: auto;
542     padding-top: 80px;
543   }
544   .hero-photo-side {
545     height: 55vw;
546     min-height: 280px;
547   }
548   .hero-content-side {
549     padding: 40px 28px 60px;
550   }
551   .hero-mini-stats { gap: 20px; }
552   .work-grid { grid-template-columns: repeat(2, 1fr); }
553   .metrics-grid { grid-template-columns: repeat(2, 1fr); }
554   .metric-item { border-right: none; border-bottom: 1px solid var(--borderS); }
555   .metric-item:nth-child(odd) { border-right: 1px solid var(--borderS); }
556   .metric-item:nth-child(3),
557   .metric-item:nth-child(4) { border-bottom: none; }
558   .case-card { grid-template-columns: 1fr; gap: 28px; padding: 32px; }
559   .case-card.reverse { direction: rtl; }
560 }
561
562 @media (max-width: 600px) {
563   .work-grid { grid-template-columns: 1fr; }
564   .book-wrap { padding: 48px 24px; }
565   .hero-float-badge:nth-child(4) { display: none; }
566 }
567
568 @media (prefers-reduced-motion: reduce) {
569   *, *::before, *::after {
570     animation-duration: .001ms !important;
571     transition-duration: .001ms !important;
572   }
573   .reveal, .stagger > * { opacity: 1; transform: none; }
574   .hero-float-badge { animation: none; }
575 }

```

Listing 34: `css/site.css`, lines 536–575

Jargon: Media query

A block of CSS applying only when a condition about the display is true. `@media (max-width: 960px)` means “apply everything here only when the window is 960 pixels wide or narrower”. Because it comes later in the file, its rules override the equivalents above.

The 960-pixel block does the real work of making this design usable on a phone. The hero collapses from two columns to one (line 540) and stops demanding the full window height (line 541). The photo column becomes 55vw tall with a 280-pixel floor (lines 545 and 546), so the photograph stays a sensible shape instead of filling a tall narrow screen.

Lines 553 to 557 fold the metrics band from four columns to two and rebuild its dividing lines: right border removed from everything, bottom border added to everything, right border restored on odd items so there is still a line down the middle, bottom border removed from items 3 and 4

so the block does not end in a stray line. That is four rules to reposition two lines, and it is the price of the border technique.

Line 558 collapses the case cards to a single column, and line 559 is the third dead rule.

The 600-pixel block collapses the work grid to one column, tightens the booking panel, and hides the third floating badge, which is the one positioned at the top of the photo column where a phone has least room.

The final block, lines 568 to 575, is the best thing in the file.

What prefers-reduced-motion is, and why this block is better than its sibling's

Operating systems let people ask for reduced motion, usually because animation causes nausea, dizziness or migraine. The browser passes that preference to the page and this block responds.

It does three things. It crushes every animation and transition to a thousandth of a millisecond, effectively instant rather than disabled, a deliberate choice because setting them to zero can stop some browsers firing the events that mark an animation as finished. Line 573 then forces every revealed and staggered element to full opacity, because those start invisible and would otherwise never appear once their transitions were neutered. That second line is the detail most implementations get wrong.

Line 574 is the addition this file makes and the sibling does not: `.hero-float-badge { animation: none; }` stops the three badges drifting outright. The blanket duration rule above would already have all but stopped them, so this is belt and braces on the one continuous, looping animation on the page, which is exactly the kind a motion-sensitive visitor would notice most.

The reduced-motion block cannot reach the counting numbers

This block governs CSS only. The four metric numbers are animated by JavaScript, counting up over 2.2 seconds, and `main.js` never checks this preference. A visitor who has asked for reduced motion still gets four numbers whirring upward.

The fix is four lines in `main.js`, given in section 7.3. This is the clearest defect in the project after the placeholder: not because it is severe, but because the intent to handle it correctly is demonstrably here, and expressed more carefully than in the sibling, and still was not carried across to the other file.

What it connects to

Named by: `new-design-reference/index.html`, line 16, and nothing else.

Names: no other file. It refers to no images and imports nothing. It names font *families*, which the HTML is responsible for downloading.

Coupled to `index.html` in three ways that no tool checks: by class name throughout, by child position for the floating badges (lines 217 to 219), and by the shared contract that JavaScript adds the class `visible`. One of those couplings is already broken, at lines 436, 437 and 559.

Coupled to `js/main.js` through exactly two class names, `scrolled` and `visible`, both of which the JavaScript adds and this file defines.

Concepts introduced

Custom properties, the cascade, flexbox, grid, pseudo-elements, pseudo-classes, keyframe animations, transitions, media queries, `clamp`, `calc`, `aspect-ratio`, `backdrop-filter`, `background-clip: text`, stacking with `z-index`, and `prefers-reduced-motion`. Each is defined at first use above and collected in the glossary.

How it breaks

- **Delete this file and the page still works**, in the sense that every word remains readable. It looks like an unstyled document, and the two-column hero becomes a stack.
- **Rename a class here without renaming it in the HTML** and that element silently loses its styling. Lines 436, 437 and 559 are a live example.
- **Insert an element before the placeholder in the HTML** and all three badges jump, because lines 217 to 219 position them by child number.
- **Remove pointer-events: none from either fade** (lines 189, 198) and those invisible layers begin swallowing clicks.
- **Remove position: relative on line 137** and the badges jump to positions measured against the whole window.
- **A missing semicolon** makes the browser discard the rest of that declaration block. Unlike a program, CSS never reports an error: it drops what it cannot parse and carries on, which is why a stylesheet bug shows up as something looking wrong rather than as a message.

Honest findings for this file

Five rules still using the agency blue after the rebrand (lines 65, 255, 256, 337, 435, 498); a sixth inherited literal in the badge background (line 205); four tokens defined and never read; three dead rules for a class the HTML does not use (lines 436, 437, 559); a doubled flex layout in the navigation; smooth scrolling implemented twice; no focus states on either button; and a reduced-motion block that is the best in either sibling and cannot reach the one animation living outside CSS.

7.3 new-design-reference/js/main.js

JavaScript, 64 lines. Everything on the page that reacts. It does three things and nothing else: the navigation bar grows a frosted background once you scroll, the four metric numbers count up from zero when they come into view, and most blocks fade upward as they arrive.

Where this came from: new-design-reference/js/main.js

Hand-written, and its own header comment says as much on line 2: **Zero external dependencies**. That claim is true and checkable: the file imports nothing, requires nothing, and the repository has no `package.json` that could supply a library.

It arrived complete in the first commit `1f7ed12` and has never been edited. It is a condensed version of the sibling repository's file: same three behaviours, same helper, same counter arithmetic, 15 lines shorter, with the comments trimmed and one behaviour dropped (the sibling watches a fifth selector this page does not have). Which was written first is not recorded and is not guessed here. **Confidence: certain for hand-written and dependency-free; the relationship to the sibling is stated as a close resemblance, not as a direction of copying.**

Why it exists

Without this file the page is complete, readable and static. Two of its three jobs are pure decoration and would not be missed. The third matters: the metric numbers are written into the HTML as the character 0 with their real values hidden in data attributes, so with this file gone the metrics band permanently advertises zero of everything.

That damage is smaller here than in the sibling, because the three hero statistics (`index.html` lines 114 to 127) state three of the same four numbers as plain text. A visitor without JavaScript

still sees “17+ projects”, “1.6M+ organic views” and “10K+ users served” in the hero, and four zeros further down. Odd, but not silent.

The single most important line in this file is in another file

`index.html` loads this script at line 393, immediately before `</body>`. Everything below runs the instant it loads, with no wrapper telling it to wait for the document. That only works because by then every element already exists. Move the `<script>` tag into the `<head>` and this file would run against an empty page, find nothing, attach nothing, and report no error at all.

Line by line

Lines 1 to 13: the header, the wrapper, and the navigation bar

```
1  /* salmanadnan.com · main.js
2     Scroll-driven animations. Zero external dependencies. */
3
4  (function () {
5     'use strict';
6
7     /* -- Nav gloss on scroll -- */
8     const nav = document.querySelector('.nav');
9     if (nav) {
10        const tick = () => nav.classList.toggle('scrolled', window.scrollY > 40);
11        window.addEventListener('scroll', tick, { passive: true });
12        tick();
13    }
```

Listing 35: `js/main.js`, lines 1–13

Lines 1 and 2 are a comment, using the same `/* */` form as CSS.

Line 4 opens a wrapper that looks like noise and is not.

Jargon: Immediately invoked function expression

Writing `(function () { ... })()`; defines a function and runs it once, straight away, without giving it a name. The point is not the running: it is the boundary. Anything declared inside is invisible outside.

Without it, the names `nav`, `animateCounter` and `observe` declared below would become global, visible to any other script on the page, and any other script using the same name would collide with them. One stray global called `observe` is exactly the kind of bug that takes an afternoon to find.

Line 5 is `'use strict'`; switching the language into a stricter mode for everything inside the wrapper.

Jargon: Strict mode

A setting making JavaScript refuse things it would otherwise silently allow. The most valuable one: without it, assigning to a name you never declared quietly creates a global variable, so a typo such as `navv = 3` succeeds and does nothing useful. In strict mode it throws an error and you find the typo immediately.

Line 8 finds the navigation bar. `document.querySelector` takes a CSS selector, exactly the syntax the stylesheet uses, and returns the first matching element or the special value `null` if there is none.

Jargon: const, and why not var

`const` declares a name whose binding cannot later be pointed at something else. It is the default choice in modern JavaScript, with `let` for names that genuinely must change. The older `var` has confusing scoping rules and appears nowhere in this file, which is a small sign of care.

Line 9 is a guard: everything happens only if `(nav)`, that is, only if something was found. This is the pattern making the whole file tolerant of a changing page.

Jargon: Truthy and falsy

JavaScript treats certain values as false in a condition: `false`, `0`, an empty string, `null` and `undefined`. Everything else counts as true. So `if (nav)` reads as “if an element was found”, because `querySelector` returns `null` when it finds nothing.

Line 10 does in one line what the sibling spreads over three, using an arrow function with an implicit return.

Jargon: Arrow function

`() => expression` is a compact way to write a function. Written without braces, as here, it returns the expression’s value automatically. The return value is discarded in this case; the compactness is the only benefit.

`classList.toggle('scrolled', condition)` adds the class when the condition is true and removes it when false. The condition is `window.scrollY > 40`, meaning the page has scrolled more than 40 pixels. The stylesheet at lines 101 to 107 defines what `scrolled` looks like, and the transition on line 99 makes the change fade rather than snap.

Line 11 registers the function to run on every scroll event. `{ passive: true }` is a performance promise to the browser: this handler will never cancel the scroll. Without the promise the browser must wait for the handler to finish before knowing whether the scroll may proceed, which can make scrolling feel sticky. It is the correct flag here and it is often forgotten.

Line 12 calls the function once immediately, handling the case where the page loads already scrolled: a refresh halfway down, or an arrival on a link ending in `#book`. Without it the bar would be transparent over content until the first scroll.

Lines 14 to 33: the counter

```

14
15  /* -- Counter animation -- */
16  function animateCounter(el) {
17      const raw    = el.dataset.target || '0';
18      const suffix = el.dataset.suffix || '';
19      const target = parseFloat(raw.replace(/[^0-9.]/g, ''));
20      const isFloat = raw.includes('.');
21      const dec    = isFloat ? (raw.split('.')[1] || '').length : 0;
22      const dur    = 2200;
23      const start  = performance.now();
24
25      const step = (now) => {
26          const p    = Math.min((now - start) / dur, 1);
27          const eased = 1 - Math.pow(1 - p, 3);
28          const val   = eased * target;
29          el.textContent = (isFloat ? val.toFixed(dec) : Math.round(val).toLocaleString()) +
          suffix;
30          if (p < 1) requestAnimationFrame(step);
31      };
32      requestAnimationFrame(step);
33  }

```

Listing 36: js/main.js, lines 14–33

This is the most substantial piece of logic in the repository and is worth taking slowly.

Line 17 reads the target. `e1.dataset.target` is how JavaScript reads the attribute `data-target` written in the HTML: strip the `data-` prefix and what remains is a property of `dataset`. The `|| '0'` supplies a default: if the attribute is missing, `dataset.target` is `undefined`, which is falsy, so the expression evaluates to `'0'`.

Line 18 does the same for the suffix, defaulting to an empty string.

Line 19 turns the text into a number. `raw.replace(/[~0-9.]/g, '')` deletes every character that is not a digit or a dot, and `parseFloat` reads what is left as a number.

Jargon: Regular expression

A pattern for matching text, written between slashes. `[0-9.]` means “a digit or a dot”; a leading caret negates it, so it means “anything that is not a digit or a dot”. The `g` after the closing slash means global: replace every occurrence, not just the first.

Line 20 asks whether the original text contained a dot, which is how the code distinguishes 1.6 from 17. Line 21 counts the digits after the dot so 1.6 displays to one decimal place throughout rather than as 1.5999999. `(raw.split('.')[1] || '')` takes everything after the dot, falling back to an empty string, and `.length` counts it.

Jargon: Ternary operator

`condition ? a : b` evaluates to `a` when the condition is true and `b` otherwise. It is an `if` statement compressed into an expression, and it appears twice in this file, on lines 21 and 29.

Line 22 fixes the duration at 2200 milliseconds. Line 23 records the start time with `performance.now()`, a clock designed for measuring durations precisely.

Lines 25 to 31 define one frame, and this is the heart of it.

`const p = Math.min((now - start) / dur, 1)` computes progress from 0 to 1. The `Math.min(..., 1)` caps it, so if the browser is busy and a frame arrives late, progress never exceeds 1 and the number never overshoots. That cap is the difference between a counter landing on 17 and one that occasionally flashes 19 before settling.

`const eased = 1 - Math.pow(1 - p, 3)` is the easing. Linear progress feels mechanical; this curve starts fast and slows dramatically near the target, which is what makes it feel like settling.

What the easing formula actually does

Take progress `p`, subtract it from 1, cube the result, subtract that from 1. At `p = 0.5`, halfway through in time, the formula gives $1 - 0.5^3 = 0.875$: the counter is already 87.5% of the way there. The whole second half of the animation covers the last eighth of the distance. That asymmetry is the entire effect.

Line 29 renders the number. If the target had a decimal point, `toFixed(dec)` formats it to that many places. Otherwise `Math.round` makes it whole and `toLocaleString()` inserts the thousands separators appropriate to the visitor’s region, so a large number reads as 1,200 for a British visitor and 1.200 for a German one. The suffix is glued on the end.

Line 30 asks for the next frame if the animation is unfinished, and line 32 starts the first.

Jargon: requestAnimationFrame

A request to run a function just before the browser's next screen repaint, about 60 times a second. Calling it repeatedly from inside itself, as here, is the standard way to run an animation: it stays in step with the display, and the browser pauses it entirely when the tab is in the background, which a timer would not.

The counter ignores the reduced-motion preference

This function animates for 2.2 seconds regardless of whether the visitor has asked their operating system to reduce motion. The stylesheet honours that preference with unusual care, going further than the sibling by explicitly stopping the floating badges, so the intent is plainly there and simply was not carried into this file.

The fix is small, and this document states it precisely so nobody has to design it:

```
1  const reduce = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
2  if (reduce) {
3      el.textContent = (isFloat ? target.toFixed(dec)
4                      : target.toLocaleString()) + suffix;
5      return;
6  }
```

Listing 37: A four-line fix. This code is a suggestion in this document and is NOT present in the repository.

Placed after line 23, that shows the final number immediately and skips the animation, which is the correct behaviour: the information is preserved and only the movement is removed.

Lines 34 to 48: the observer helper

```
34
35  /* -- Intersection observer helper -- */
36  function observe(selector, className, threshold, cb) {
37      const els = document.querySelectorAll(selector);
38      if (!els.length) return;
39      const io = new IntersectionObserver((entries) => {
40          entries.forEach((e) => {
41              if (!e.isIntersecting) return;
42              e.target.classList.add(className || 'visible');
43              if (cb) cb(e.target);
44              io.unobserve(e.target);
45          });
46      }, { threshold: threshold || 0.18 });
47      els.forEach((el) => io.observe(el));
48  }
```

Listing 38: js/main.js, lines 34–48

This is the best-designed thing in the repository: a small general-purpose helper that the three lines following it reuse for three different jobs.

Jargon: IntersectionObserver

A browser facility that watches elements and reports when they enter or leave the visible area. Before it existed, the only way to do this was to listen for every scroll event and measure each element's position by hand, which ran hundreds of times a second and made pages stutter. The observer does the work outside the main thread and reports only when something changes.

Line 37 finds every matching element. `querySelectorAll` returns a list, which may be empty, rather than `null`. Line 38 guards against an empty list and returns early: `!els.length` reads as

“if the count is zero”, because 0 is falsy and ! inverts it.

Lines 39 to 46 create the observer. Its callback receives one entry per element whose visibility changed, and for each it skips anything leaving rather than entering, adds the class (defaulting to `visible`), runs the optional extra callback, and stops watching that element.

That last step is the one that matters most.

Why it stops watching

Every reveal in this design is meant to happen once. Without `unobserve`, an element scrolling out of view and back would fire again, and for the metrics that would mean the numbers restarting their count every time the visitor scrolled past. Unobserving also releases the browser’s bookkeeping, so what starts as a dozen watched elements steadily becomes none.

Line 46 sets the threshold, defaulting to 0.18: the callback fires once 18% of the element is visible rather than at the first pixel. Waiting for a fraction is what makes the reveal feel connected to the scroll instead of triggering just off screen.

The defaults cannot express a falsy value

`threshold || 0.18` substitutes the default whenever the argument is falsy, and 0 is falsy. A caller asking for a threshold of 0, meaning “fire the moment a single pixel appears”, would silently get 0.18. The same objection applies to `className || 'visible'`, though an empty class name is not a sensible request.

No caller passes 0, so nothing is broken today. It is a latent trap worth recognising, and the modern remedy is the `??` operator, which substitutes only for `null` and `undefined` and leaves 0 alone.

Lines 49 to 52: the three things being watched

```
49
50   observe('.reveal', 'visible', 0.15);
51   observe('.stagger', 'visible', 0.15);
52   observe('.metric-num[data-target]', 'counted', 0.5, animateCounter);
```

Listing 39: `js/main.js`, lines 49–52

Three calls, and the page’s entire reveal behaviour. The first two add the class `visible` to anything carrying `reveal` or `stagger`, both at a 0.15 threshold, and the stylesheet does everything after that.

The third is different. Its selector `.metric-num[data-target]` matches elements carrying the class *and* the attribute, a small piece of defensiveness: a metric element without a target is skipped rather than animated to `NaN`. Its threshold is 0.5, so the numbers start counting only when the band is half visible.

Note the callback is passed as a bare function name, `animateCounter`, rather than wrapped in another arrow function. That is cleaner than the sibling, which writes `(e1) => { animateCounter(e1); }` for the same effect.

The class `counted` is added and styled nowhere

The third call passes `'counted'` rather than `'visible'`, so each metric number gains a class no CSS rule and no other JavaScript ever reads. It is harmless and could serve as a marker for a developer inspecting the page, but nothing uses it. Note also that the metric numbers are children of `.metrics-grid`, which is a `.stagger`, so they are already faded in by the second call: this class is genuinely spare.

Lines 53 to 64: smooth anchor scrolling, and the close

```
53
54  /* -- Smooth anchor scroll -- */
55  document.querySelectorAll('a[href^="#"]').forEach((a) => {
56    a.addEventListener('click', (e) => {
57      const t = document.querySelector(a.getAttribute('href'));
58      if (!t) return;
59      e.preventDefault();
60      t.scrollIntoView({ behavior: 'smooth', block: 'start' });
61    });
62  });
63
64  }());
```

Listing 40: js/main.js, lines 53–64

Line 55 finds every link whose destination starts with a hash. `a[href^="#"]` is an attribute selector and `^=` means “starts with”. On this page that matches three links: the navigation’s “Book a call”, the hero’s “Book a call”, and the hero’s “See the work”.

Line 57 looks up the destination using the link’s own `href` as a selector, which works because `#book` is valid CSS meaning “the element with id book”.

Line 58 returns early if the destination does not exist, leaving the browser to handle the click normally. Line 59 then cancels the browser’s default jump, but only after the destination is confirmed, which is the correct order: cancel first and a broken link would do nothing at all instead of at least trying.

Line 60 performs the scroll, with `block: 'start'` aligning the destination to the top of the window.

This whole block is redundant, and it carries a latent crash

`css/site.css` line 28 already sets `scroll-behavior: smooth` on the document, making every anchor link glide with no JavaScript at all. These ten lines reproduce that.

They also introduce a failure the CSS version does not have. A link written `href="#"`, a common way to write a placeholder link, would reach line 57 as `document.querySelector('#')`. That is not a valid selector, so the browser throws a `SyntaxError`, and because it happens inside a click handler it would break that click and leave an error in the console.

No such link exists today, so this is latent rather than live. The honest summary is that the safest change to this file would be to delete lines 53 to 62 entirely: it would remove a redundancy and a hazard in one edit, and nothing visible would change.

Line 64 closes and immediately runs the wrapper opened on line 4.

What it connects to

Named by: `new-design-reference/index.html`, line 393, and nothing else.

Names: no file at all. It imports nothing and fetches nothing.

Its entire interface with the rest of the project is six strings, worth listing because they are the contract nothing checks:

String	Used at	Where it must also exist
<code>.nav</code>	line 8	<code>index.html</code> line 23, <code>site.css</code> line 91
<code>scrolled</code>	line 10	<code>site.css</code> line 101
<code>.reveal</code>	line 50	<code>index.html</code> (five places), <code>site.css</code> line 75
<code>.stagger</code>	line 51	<code>index.html</code> (two places), <code>site.css</code> line 82
<code>visible</code>	lines 50–51	<code>site.css</code> lines 80 and 88
<code>.metric-num</code>	line 52	<code>index.html</code> (four places), <code>site.css</code> line 407

Table 4: Every point of contact between this file and the other two. Break any one string in one place and the page keeps working, minus one behaviour, with no error reported anywhere.

How it breaks

- **Load it in the `<head>` instead of at the end of the body** and all three behaviours vanish silently, since every lookup runs immediately and finds nothing.
- **Rename a class in the HTML** and the corresponding behaviour stops, silently, because of the early-return guards.
- **Delete this file** and the page is static, with four zeros in the metrics band. The hero statistics still read correctly, which limits the damage.
- **Add a link with `href="#"`** and clicking it throws a `SyntaxError` from line 57.
- **A visitor who has asked for reduced motion** still gets the counting animation, because this file never checks.
- **An ancient browser without `IntersectionObserver`** would throw on line 39 and lose all three reveal behaviours, which would leave every `.reveal` element permanently invisible were it not for the CSS reduced-motion block. Every browser in current use supports it, so this is a historical note rather than a live concern.

Honest findings for this file

The counter ignores `prefers-reduced-motion`; the smooth-scroll block duplicates a CSS feature and carries a latent `SyntaxError`; the `counted` class is inert; and the `||` defaults on lines 42 and 46 cannot express a falsy argument. Against those: the file is genuinely well written, with consistent guards, a sensible shared helper, correct use of `unobserve` and `passive`, no dependencies, and slightly tighter than its sibling in two places.

7.4 new-design-reference/netlify.toml

TOML, 11 lines. Instructions for a hosting service called Netlify: how to publish the folder, and four security headers to send with every response.

Where this came from: new-design-reference/netlify.toml

Hand-written, added in the first commit `1f7ed12` and never edited. It is **byte for byte identical** to the sibling repository's copy, which was verified by comparing the two files directly: `diff` reports no differences at all. Every other file in this pair differs somewhere; this one does not.

That is itself the evidence for how it got here: it is the one file in the design that carried no site-specific content, so it was reused without needing a single edit. The structure follows Netlify's own documented configuration format. **Confidence: certain for hand-written and for the identity with the sibling, both checked directly; no specific template is claimed.**

Why it exists

Today, for no reason at all: nothing reads it.

The honest reading of the history is that the site was first expected to be hosted on Netlify, a common choice for a static page like this, and the plan changed two commits later. Commit 692e162 on 2026-07-28 is titled in part “VPS rsync deploy”, which is the moment Netlify stopped being the destination. This file was never removed.

Jargon: TOML

A file format for configuration designed to be obvious to read. A name in square brackets starts a section; `name = "value"` sets a setting. Double square brackets, as on line 5, mean “another entry in a list of these”, so a file can carry several header rules.

Line by line

```
1 [build]
2   publish = "."
3   command = ""
4
5 [[headers]]
6   for = "/*"
7   [headers.values]
8     X-Frame-Options = "DENY"
9     X-Content-Type-Options = "nosniff"
10    Referrer-Policy = "strict-origin-when-cross-origin"
11    Permissions-Policy = "camera=(), microphone=(), geolocation=()"
```

Listing 41: netlify.toml, lines 1–11

Lines 1 to 3 describe how to build the site. `publish = "."` means publish this directory itself, and `command = ""` means there is no build step to run first. Those two lines are the entire “no build step” decision, stated to a hosting provider.

Line 5 opens a headers rule and line 6 applies it to `/*`, every path on the site.

Jargon: HTTP header

An extra line of information a web server sends alongside a file, which the visitor never sees. Some describe the file; the four below are instructions to the browser about what it is allowed to do with the page.

Line 8, `X-Frame-Options = "DENY"`, forbids any other site from embedding this page inside a frame. That defends against clickjacking, where an attacker loads your real page invisibly over their own and tricks a visitor into clicking your buttons while believing they are clicking something else.

Line 9, `X-Content-Type-Options = "nosniff"`, tells the browser to believe the declared type of every file rather than guessing from its contents. Guessing is a historical browser behaviour that can be manipulated into treating an uploaded image as a script.

Line 10, `Referrer-Policy = "strict-origin-when-cross-origin"`, limits what is disclosed when a visitor follows a link away. Within this site the full address is passed; to another site only the domain; to an insecure site nothing at all. This is the modern default and a sensible privacy choice.

Line 11, `Permissions-Policy`, switches off three device capabilities for this page entirely: camera, microphone and location. The empty parentheses mean “nobody, not even this page”. The page is static text and gradients and needs none of them, so turning them off means a future compromise cannot quietly request them.

These four headers are good practice and they are not in effect

Every one is a sensible defensive choice, and the file they live in is read by a service this project does not use. The page is served by Caddy on the agency VPS, which never sees this file.

Moving these four lines into the Caddy configuration on the server would take a few minutes and make them real. As things stand, the repository looks like it has thought about security headers and the live site has none of them.

What it connects to

Nothing in this repository refers to it. Netlify appears nowhere else in the tree and no workflow reads it. The deploy workflow does not even exclude it, so if that workflow were restored this file would be copied to the VPS where Caddy would ignore it.

Concepts introduced

TOML as a configuration format, and the four HTTP response headers above. No concept here appears anywhere else in the repository, which is another way of saying this file is disconnected from everything.

How it breaks

It cannot break, because nothing reads it. The realistic failure is the opposite: a future maintainer reads this file, concludes the site sends these headers, and builds on an assumption that is false.

An unused configuration file is worse than no configuration file

This is a small file and a real trap. It states four security guarantees the live site does not provide, and nothing in the repository indicates it is inert. Either delete it, or move its contents to the Caddy configuration and delete it. Leaving it is the only option with no upside.

The identical file sits in the sibling repository with the identical problem, so fixing one should mean fixing both.

7.5 new-design-reference/assets/ASSETS.md

Markdown, 32 lines. A checklist of the eleven files somebody would need to produce before this design could be considered finished, plus three notes about placeholder content to replace.

Where this came from: new-design-reference/assets/ASSETS.md

Hand-written, added in the first commit `1f7ed12` alongside the design it describes, and never edited. Written by the same person in the same sitting as the HTML, which is evident from its references to the exact class names used there (`.hero-photo-placeholder`, `wt1` through `wt6`, `cv1`, `cv2`) and from the fact that line 9 quotes, character for character, the `<img>` tag already written into the HTML's banner comment at line 34. **Confidence: high, from the commit history and the internal references.**

Why it exists

It is a handover note. The design was built with placeholder visuals, and this file records precisely what would replace each one and what HTML change each replacement needs. As an artefact it is genuinely good practice: it converts “this is not finished” from a feeling into a list.

It is also the document that makes the state of this project unambiguous. The placeholder in the hero could be argued to be a design choice; a checklist listing the photograph as *required before launch* cannot.

Jargon: Markdown

A way of writing formatted text in a plain file, using punctuation instead of menus: # starts a heading, backticks mark code, and pipe characters draw a table. It is designed to be readable as plain text and is the standard format for notes kept alongside code.

Line by line

```

1 # Assets checklist: salmanadnan.com
2
3 Drop real assets here, then update the HTML as noted.
4
5 ## Required before launch
6
7 | File | Size | Where used | HTML change needed |
8 |---|---|---|---|
9 | `salman.jpg` | 1200px tall min, 2:3 ratio | Hero photo left side | Replace `.hero-photo-
  placeholder` div with `![Salman Adnan](/assets/salman.jpg)` |
19 | `og-image.png` | 1200x630px | Social share | Update `og:image` meta tag |

```

Listing 42: ASSETS.md, lines 1–19

Line 1 is the title and line 3 the instruction: put the real files in this folder, then change the HTML as the table says.

Lines 7 and 8 open a table, line 8 being the separator row Markdown requires between a table's headings and its body.

The eleven rows are the substance, and reading them tells you exactly what the design is pretending to have.

The photograph is first, and it is the one that matters. Line 9 specifies it precisely: at least 1200 pixels tall, in a 2:3 portrait ratio, and it gives the complete replacement tag including `object-fit: cover` so the image fills the frame without distorting, and `object-position: top` so a face stays in shot when the frame crops. Whoever wrote this knew exactly what was needed and did not have the file.

Six project thumbnails at 800 by 450, which is exactly the 16 by 9 shape the stylesheet reserves with `aspect-ratio` at line 341. Line 11 gives the pattern for the remaining five rather than repeating it, which is why rows 12 to 15 have an empty final column.

Two case-study images at 1200 by 750, matching the 16/10 aspect ratio set at `site.css` line 441. That the two ratios differ, and that each listed asset matches its own, is a small sign of care.

A favicon and a social preview image, each with the exact tag needed to install it.

None of the eleven exist, and the live site has at least one of them

The `assets/` folder contains exactly one file: this checklist. Every asset it calls “required before launch” is absent, so the design was never finished to the standard its own author set. The comparison that makes this sharp: fetching the live `salmanadnan.com` on 2026-08-11 found `<meta property="og:image" content= "https://salmanadnan.com/assets/og.png">` along with explicit width and height tags. So the social preview image listed here as required does exist, for the build that replaced this one. This design was superseded by a version that finished the checklist.

```

20
21 ## Numbers to update
22
23 Search for `data-target` in `index.html` (metrics band) and replace all four values with real
  numbers.
24 Search for the three `hero-mini-num` divs in the hero and update those too.
25
26 ## Booking link
27
28 Search for `YOUR_CAL_LINK` in `index.html` and replace with the actual booking URL.
29
30 ## Client names
31
32 Three `<!-- UPDATE -->` comments in the work grid and case study sections mark placeholder
  client entries.

```

Listing 43: ASSETS.md, lines 20–32

Line 23 is the metrics instruction, and line 24 is the one worth noticing.

This file knows about the duplication and tells you to fix it in both places

Line 23 says to search for `data-target` in the metrics band. Line 24 then says to search for the three `hero-mini-num` divs and update those too.

That is the author documenting, at the time of writing, that the same three numbers live in two places in two formats, as section 7.1 described. It was a recognised cost, accepted deliberately, and written down. That is a materially different thing from an accident, and it is why this document treats the duplication as a design trade rather than a defect.

The sibling repository has no equivalent note because it has no hero statistics, and consequently no fallback when JavaScript does not run.

Line 28 tells the reader to search for `YOUR_CAL_LINK`.

Two of this file’s three instructions are stale

Searching `index.html` for `YOUR_CAL_LINK` finds nothing. The literal placeholder was replaced on 2026-07-28 by commit 014406e, which set the link to `cal.com/salman-adnan/meeting`, a real personal calendar. So somebody following this instruction searches, finds nothing, and cannot tell whether the job is done or the instruction is wrong. Here the job *is* done, unlike in the sibling repository where the same stale instruction hides a link still pointing at another project’s calendar.

Line 32 is stale in a different way. It refers to “three `<!-- UPDATE -->` comments in the work grid and case study sections” marking placeholder client entries. There are no such comments anywhere in `index.html`: its `UPDATE` notes live in the banner comments at the top of each section and none of them is about client names. Either they were removed or they were never written.

Only the metrics instruction on lines 23 and 24 is verifiably still accurate, and the `UPDATE`

comment at `index.html` line 256 agrees with it.

What it connects to

It describes `new-design-reference/index.html` and `new-design-reference/css/site.css` by name and by class, and it is the only file in the repository documenting the project's intent. Nothing reads it: it is for people.

The deploy workflow explicitly excludes it from publication (line 82 of the workflow), which is the right instinct, and that exclusion no longer matches its path. See section 7.6.

Concepts introduced

Markdown tables; `object-fit` and `object-position`, the two properties that control how an image fills a frame it does not exactly match; and the standard sizes for a favicon (32 by 32) and an Open Graph preview image (1200 by 630).

How it breaks

As a document it cannot break. As a source of truth it already has: two of its three instructions describe a state of the code that no longer exists or never did, which is the characteristic failure of documentation kept beside code and not updated with it.

Honest findings for this file

Two stale instructions (lines 28 and 32); eleven assets specified in useful detail and never delivered; and, to its credit, one instruction (lines 23 and 24) that is still exactly right and documents a known duplication rather than hiding it.

7.6 `.github/workflows-disabled/deploy.yml.disabled`

YAML, 103 lines. The automation that used to publish this site: it watched for pushes, checked for a marker in the commit message, copied the files to the agency's server over an encrypted connection, then checked that the site still answered. It is currently switched off.

Where this came from: `.github/workflows-disabled/deploy.yml.disabled`

Hand-written, added in commit `05210af` on 2026-07-28 at the path `.github/workflows/deploy.yml`, rewritten in `692e162` the same day when the deploy target changed from Netlify to the VPS, fixed four times on 2026-07-29, and finally renamed to its present path in `0dd839e`.

It is **nearly identical to the sibling repository's workflow**: comparing the two files directly shows differences on exactly six lines, all of them the site's own name or path (the header comment, the serving path, the workflow name, the `rsync` destination and the verification URL). Every mechanism, every retry loop and every flag is the same.

This repository is where the evidence for those mechanisms lives. Its history contains two commits explaining the retry loops by name (`6b1aa25`, `6fcffee`); the sibling has the loops and no such commits. The `[deploy]` convention itself is workspace-wide and predates both.

Confidence: certain for hand-written and for the rename history, both recorded in git; certain for the six-line difference, checked directly.

Why it exists

To remove a manual step. Without it, publishing means somebody connecting to the server and copying files by hand, which is slow and easy to get wrong. With it, publishing is a consequence of writing a commit message in a particular way.

Jargon: YAML

A file format for configuration where structure is expressed by indentation rather than brackets. Two spaces of indent mean “this belongs to the line above”. It is unforgiving about indentation, which is its main drawback: one misplaced space changes the meaning.

Jargon: GitHub Actions, and a workflow

A service running jobs on GitHub’s computers when something happens in a repository. The jobs are described in a workflow file in `.github/workflows/`. Each job runs on a fresh, empty machine, which is why the file below installs its own SSH key every time.

Line by line

Lines 1 to 19: the header comment and the trigger

```

1 # salmanadnan.com: deploy to VPS when commit subject starts with [deploy].
2 #
3 # A push to main always checks the marker. It DEPLOYS only when the commit
4 # subject line STARTS with [deploy], matching every other agency repository.
5 # Mentioning the marker anywhere else in the message does nothing.
6 #
7 # The site is served by Caddy from /root/salmanadnan.com on the VPS.
8 # Caddy picks up the new files immediately, no restart needed.
9 #
10 # Required repository secrets:
11 #   VPS_HOST      46.202.194.85
12 #   VPS_USER      deploy
13 #   VPS_SSH_KEY   private key for that user
14 name: Deploy salmanadnan.com
15
16 on:
17   push:
18     branches: [main]
```

Listing 44: `deploy.yml.disabled`, lines 1–19

Lines 1 to 13 are a comment block, and it is the best documentation in the repository. In YAML a `#` starts a comment. It states the marker rule twice and in capitals, which tells you it had caught somebody out; it names the serving path and notes that Caddy needs no restart; and it lists the three secrets required.

The server’s address is written in the comment explaining it is a secret

Line 11 records `VPS_HOST 46.202.194.85` in plain text immediately under the heading “Required repository secrets”. Whatever the value of keeping the address in GitHub’s secret store, writing it here defeats it.

Proportion matters: an IP address is not a credential, this repository is private, and the machine is publicly reachable anyway. This document confirmed independently that `salmanadnan.com` resolves to exactly this address today, which incidentally makes the comment currently accurate. The genuine cost is that it is permanent in the git history, so it becomes misleading rather than merely redundant if the server is ever replaced. The private key, which is what actually matters, is correctly absent.

Line 14 names the workflow, the label shown in GitHub’s interface.

Lines 16 to 18 set the trigger: every push to `main`, and nothing else. Not on pull requests, not on a schedule, not manually. That last absence is worth noting: there is no `workflow_dispatch`, so there is no way to publish without making a commit.

Lines 20 to 38: the marker check

```

20 jobs:
21   check-deploy:
22     name: Check Deploy Marker
23     runs-on: ubuntu-latest
24     outputs:
25       should_deploy: ${ steps.check.outputs.should_deploy }
26     steps:
27       - id: check
28         env:
29           COMMIT_MSG: ${ github.event.head_commit.message }
30         run: |
31           SUBJECT="${COMMIT_MSG%%$'\n'*}"
32           if [[ "$SUBJECT" == "[deploy]*" ]]; then
33             echo "should_deploy=true" >> "$GITHUB_OUTPUT"
34             echo "Deploy marker found on the subject line."
35           else
36             echo "should_deploy=false" >> "$GITHUB_OUTPUT"
37             echo "No deploy marker: skipping deploy."
38           fi

```

Listing 45: `deploy.yml.disabled`, lines 20–38

The first job exists only to answer a yes-or-no question, and runs on a fresh Ubuntu machine to do it. Lines 24 and 25 declare that it produces an output called `should_deploy`, which the second job will read.

Line 28 passes the commit message in as an environment variable rather than pasting it into the script text. That distinction is a genuine security measure.

Why the commit message goes through an environment variable

If line 31 had been written `SUBJECT="${ github.event.head_commit.message }"`, GitHub would paste the commit message directly into the script before the shell ever saw it. Anyone who could write a commit message could then write shell commands into it and have them run on the machine holding the deploy key.

Passing it as an environment variable and reading it as `$COMMIT_MSG` means the message is data, never code. This is a well-known class of vulnerability in GitHub Actions, and this file gets it right.

Line 31 extracts the subject line. `${COMMIT_MSG%%$'\n'*}` is shell syntax meaning “delete the longest match of a newline followed by anything, from the end”, leaving everything before the first newline.

Line 32 is the test. `"$SUBJECT" == "[deploy]*"` compares with a trailing wildcard, which is why the marker must *begin* the subject.

Lines 33 and 36 write the answer to `$GITHUB_OUTPUT`, a file GitHub reads to collect a step’s outputs. Lines 34 and 37 print a human-readable explanation into the log, a small kindness to whoever is wondering why nothing deployed.

Lines 39 to 63: starting the deploy, and the SSH setup

```

39
40   deploy:
41     name: Deploy to VPS
42     runs-on: ubuntu-latest
43     needs: check-deploy

```

```

44     if: needs.check-deploy.outputs.should_deploy == 'true'
45     steps:
46       - uses: actions/checkout@v4
47
48       - name: Authorize SSH
49         env:
50           KEY: ${ secrets.VPS_SSH_KEY }
51           HOST: ${ secrets.VPS_HOST }
52         run: |
53           mkdir -p ~/.ssh && chmod 700 ~/.ssh
54           printf '%s\n' "$KEY" > ~/.ssh/deploy_key
55           chmod 600 ~/.ssh/deploy_key
56           for i in 1 2 3 4 5; do
57             if ssh-keyscan -T 10 -H "$HOST" >> ~/.ssh/known_hosts 2>/tmp/keyscan.err; then
58               [ -s ~/.ssh/known_hosts ] && break
59             fi
60             echo "ssh-keyscan attempt $i failed: $(cat /tmp/keyscan.err 2>/dev/null)"
61             sleep 5
62           done
63           [ -s ~/.ssh/known_hosts ] || { echo "::error::could not fetch the host key"; exit 1;
    }

```

Listing 46: `deploy.yml.disabled`, lines 39–63

Line 43 makes this job wait for the first, and line 44 runs it only if the answer was yes. Everything below is skipped entirely on an ordinary commit.

Line 46 checks out the repository onto the machine using GitHub’s own action. Without this the machine would be empty.

Lines 53 to 55 install the private key: create the `.ssh` folder, write the key into it, and set permissions so only the owner can read it. `chmod 600` is not optional; SSH refuses to use a key file other users can read, and its error message when it does is famously unhelpful.

Jargon: SSH key pair, and the host key

Connecting securely involves two separate keys and confusing them is common. The *deploy key* here is the private half of a pair proving *we* are allowed in; its public half sits on the server. The *host key* is the server’s own identity, which the client checks to be sure it is talking to the right machine and not an impostor. Lines 53 to 55 install the first; lines 56 to 63 fetch the second.

Lines 56 to 62 are the retry loop, and this repository records exactly why it exists. `ssh-keyscan` asks the server for its host key with a 10-second timeout, appending the result to `known_hosts` and any complaint to a temporary file. `[ -s ~/.ssh/known_hosts ]` then tests that the file is non-empty, because `ssh-keyscan` can succeed as a command while producing nothing at all. That distinction is the subtle part: checking the exit code alone would not have been enough.

Two commits name this problem outright

Commit `6b1aa25` is titled “Retry after transient keyscan failure” and `6fcffee` “Retry, runner IP lottery”. Read together they are a complete diagnosis: GitHub’s runners come from a wide pool of addresses, some of them were being refused or timing out, and the same push succeeded on a second attempt.

This is the clearest example in the whole pair of repositories of a challenge with a scar. The sibling has the identical retry loop and no commit explaining it, so anybody reading only that repository would see defensive code with no stated cause.

Line 63 fails the whole job with a message if five attempts produced nothing. `::error::` is GitHub’s syntax for marking a log line as an error so it surfaces in the interface.

Why fetching the host key each time is a real trade-off

Fetching a server's identity at the moment you connect, and trusting whatever comes back, means you are not verifying anything: an attacker positioned between the runner and the server could supply their own. The secure alternative is to store the expected host key as a secret and compare.

That is a real weakness and it is a widespread convention, and another project in this workspace, `passbolt-digitalise`, does it the stricter way by keeping a `.github/known_hosts` file in the repository. Two approaches to the same problem inside one workspace is worth knowing about.

Lines 64 to 83: copying the files

```

64
65     - name: Sync files
66     env:
67         HOST: ${ secrets.VPS_HOST }
68         USER: ${ secrets.VPS_USER }
69     run: |
70         retry() {
71             for attempt in 1 2 3; do
72                 "$@" && return 0
73                 echo "attempt $attempt failed: $*"
74                 [ "$attempt" -lt 3 ] && sleep $(( attempt * 15 ))
75             done
76             return 1
77         }
78         retry rsync -az --no-owner --no-group --omit-dir-times --delete \
79             -e "ssh -i ~/.ssh/deploy_key -o BatchMode=yes -o ConnectTimeout=25" \
80             --exclude '.git' \
81             --exclude '.github' \
82             --exclude 'assets/ASSETS.md' \
83             ./ "$USER@$HOST:/root/salmanadnan.com/"

```

Listing 47: `deploy.yml.disabled`, lines 64–83

Lines 70 to 77 define a small shell function running whatever it is given up to three times, waiting longer between attempts: 15 seconds, then 30. "\$@" means “all the arguments, run as a command”. This is the second retry loop, and like the first it is evidence of a step that failed in practice.

Line 78 is the copy itself. Each flag earns its place:

Flag	What it does
<code>-a</code>	archive mode: recurse, and preserve timestamps, permissions and links
<code>-z</code>	compress during transfer
<code>-no-owner</code>	do not try to set file ownership on the far end
<code>-no-group</code>	do not try to set group ownership either
<code>-omit-dir-times</code>	do not try to set directory timestamps
<code>-delete</code>	remove files on the server that no longer exist here

Table 5: The `rsync` flags, and why each is present.

The three `-no` and `-omit` flags all work around the same thing: the `deploy` user does not own the target directory and cannot change ownership or directory timestamps. Without them `rsync` reports errors and fails. These flags are the surviving trace of the problem commit `431ebbf` was fixing.

`-delete` deserves respect: it makes the server an exact mirror, so removing a file here removes it there. That is usually what you want, and it is also the flag that turns a mistaken source path into a wiped site.

Line 79 tells `rsync` how to connect: use this key, never prompt for a password (`BatchMode=yes`, essential because there is no human to answer), and give up after 25 seconds.

Lines 80 to 82 exclude three things: the git history, the workflow folder, and the internal checklist.

Line 83 is the source and destination. `./` means the current directory's *contents*, and the trailing slash is significant: without it `rsync` would copy the directory itself into the target, producing a nested folder.

This line would now take down the live site

`./` was the right source when `index.html` sat at the repository root. Since commit `0dd839e` moved everything one level down, `./` contains a single folder called `new-design-reference`. Restoring this workflow unchanged would copy that folder to the server and, because `-delete` is set, remove the real site's files in the same operation.

This is not hypothetical harm. Section 1.2 established that `salmanadnan.com` is live, answering with 200, and resolves to the very address on line 11. The correct source after the move would be `new-design-reference/`.

The same commit that moved the files disabled this workflow, so the two are consistent; the trap exists only for somebody who re-enables one without reading the other.

The exclusion for the checklist no longer matches

`-exclude 'assets/ASSETS.md'` was correct when that path was correct. The file now lives at `new-design-reference/assets/ASSETS.md`, which this pattern does not match, so a restored workflow would publish the internal checklist, including its instruction to drop a photograph in, to the public internet.

Lines 84 to 103: verifying, and cleaning up

```

84
85     - name: Verify
86       env:
87         HOST: ${ secrets.VPS_HOST }
88         USER: ${ secrets.VPS_USER }
89       run: |
90         for i in $(seq 1 12); do
91           code=$(ssh -i ~/.ssh/deploy_key -o BatchMode=yes "$USER@$HOST" \
92             'curl -sf -o /dev/null -w "%{http_code}" --max-time 10 https://salmanadnan.com/'
93         || echo 000)
94           case "$code" in
95             2*|3*) echo "Site answering with $code after ${i} check(s)."; exit 0 ;;
96             esac
97           echo "waiting, got $code (${i}/12)"
98           sleep 5
99         done
100        echo "::warning::site did not respond 2xx/3xx; check Caddy on the VPS"
101
102     - name: Clean up key
103       if: always()
104       run: rm -f ~/.ssh/deploy_key

```

Listing 48: `deploy.yml.disabled`, lines 84–103

The verification step asks, up to twelve times over about a minute, whether the site is answering. It does this by connecting to the server and running `curl` *there* rather than from the runner, which tests the server's own view of itself.

The `curl` flags: `-s` quiet, `-f` treat an error status as a failure, `-o /dev/null` throw the page away, `-w "%{http_code}"` print just the status number, and a 10-second limit. The `|| echo 000` supplies a fallback so a failed connection yields the harmless string 000 rather than an empty one.

Line 93 accepts any status beginning with 2 or 3, meaning success or redirect, and exits successfully. That tolerance is well judged and this document can confirm it matters: <http://salmanadnan.com/> currently answers 308, a permanent redirect to the secure version, so a check accepting only 200 could fail on a perfectly healthy site.

A failed verification is only a warning

Line 99 uses `::warning::` rather than `::error::`, and the step does not exit with a failure code. So if the site never came back, the workflow would still finish green with a warning buried in the log.

Given that the preceding step ran `rsync -delete` against a live site, this is the one moment you would want a loud red failure and an email. It is the clearest correctness weakness in the workflow, and it is one word to fix.

Lines 101 to 103 delete the private key from the runner. `if: always()` makes this run even when an earlier step failed, which is the point: a key must not be left behind because something else went wrong. In practice GitHub destroys the machine anyway, so this is belt and braces, and it is the right instinct.

What it connects to

Reads: three GitHub secrets (`VPS_SSH_KEY`, `VPS_HOST`, `VPS_USER`) and the commit message of the push that triggered it.

Writes: the contents of `/root/salmanadnan.com/` on the VPS at 46.202.194.85, which Caddy serves at <https://salmanadnan.com/>.

Excludes: `.git`, `.github` and `assets/ASSETS.md`.

Runs when: never, currently. It sits in `.github/workflows-disabled/` with the extension `.disabled`, and GitHub only reads `.github/workflows/*.yml`.

Concepts introduced

YAML; GitHub Actions jobs, steps, outputs and secrets; environment variables as a defence against injection; SSH key pairs and host keys; `rsync` and its archive and delete semantics; retry loops with growing backoff; and the `[deploy]` marker convention shared across this workspace.

How it breaks

- **Restored unchanged**, it copies the wrong directory and, with `-delete`, takes down the live `salmanadnan.com`.
- **A missing or wrong `VPS_SSH_KEY`** fails at the `rsync` step after three attempts. Commit `cfa65d7` fixed this.
- **A deploy user without permission to traverse into `/root`** fails the same way. Commit `431ebbf` fixed this, and it is the least obvious of the failures: the user can hold the key, connect successfully, and still be unable to reach the target directory.
- **GitHub's runner being refused by the server** fails the host-key fetch. Commits `6b1aa25` and `6fcffee` are both this.
- **A commit whose subject does not start with `[deploy]`** does nothing at all, by design, and the log says so.
- **The site failing to come back** produces a warning and a green tick, which is the failure mode worth fixing first.

Honest findings for this file

The server address is recorded in a comment describing it as a secret; the `rsync` source path and the `ASSETS.md` exclusion are both stale after the move, and here the consequence is a live site going down rather than a parked one; the host key is trusted on first sight rather than pinned, unlike `passbolt-digitalise` in the same workspace; and a failed post-deploy check produces a warning rather than a failure. Against those: the commit message is handled safely as data rather than as code, both retry loops are well built and documented by the commits that caused them, the status check correctly tolerates redirects, and the key is cleaned up unconditionally.

8 Generated and third-party files

There are none. All six tracked files were written by hand. No lock file, no minified bundle, no vendored dependency, no binary asset, and no generator banner. Checked rather than assumed:

```
1 $ git grep -lIE '(DO NOT EDIT|auto-?generated|@generated|Generated by)'
2 (no output)
```

Listing 49: The check that was run, and its result

So the tier 2 count in Appendix A is zero and every file is walked line by line.

9 Configuration and secrets

Nothing is configurable at run time: no settings file, no environment file, no `.env.example`. To change what the page says, you edit the HTML.

Three values behave like configuration and are written into the source:

Value	Where	What it controls
<code>https://cal.com/salman-adnan/meeting</code>	<code>index.html</code> line 362	the booking calendar
<code>salman@digitalise.agency</code>	<code>index.html</code> line 386	the footer contact address
The nineteen tokens	<code>css/site.css</code> lines 5–25	the whole colour scheme

Table 6: Values that behave like settings but are written into the source.

9.1 Secrets

The repository contains **no passwords, keys or tokens**, and should not: a page running entirely in a visitor’s browser can keep no secret.

Three are referred to by the workflow and stored in GitHub rather than here: `VPS_HOST`, `VPS_USER` and `VPS_SSH_KEY`.

The server’s IP address is written in a comment

Line 11 of the workflow records `VPS_HOST 46.202.194.85` in a comment directly under the heading “Required repository secrets”. Storing the value in a comment defeats storing it as a secret.

Proportion matters: an IP address is not a credential, this repository is private, and the machine is reachable from the internet anyway; this document confirmed independently that `salmanadnan.com` resolves to exactly that address. The genuine cost is that it is now permanent in the git history, so it becomes misleading rather than merely redundant if the server is ever replaced. The private key, which is what actually matters, is correctly absent.

10 Dependencies

No build-time dependencies at all: no `package.json`, no `requirements.txt`, nothing to install.

Two run-time dependencies, both services on other people’s computers, both reached by the visitor’s browser rather than by the project:

Dependency	Reached from	What it is for
Google Fonts cal.com	<code>index.html</code> lines 13–15 <code>index.html</code> line 362	downloads the Space Grotesk and Inter typefaces shows the calendar when “Book a call” is pressed

Table 7: The two external services the page depends on.

Fonts are loaded from Google, which the agency’s main site deliberately does not do

The agency’s main website (`digitalise-website` in this workspace) self-hosts its fonts. This page fetches them from `fonts.googleapis.com` on every visit, so every visitor’s browser contacts Google, which is a privacy and consent question in some jurisdictions, and the page’s appearance depends on a third party staying up.

It does degrade gracefully: both font definitions list fallbacks (`ui-sans-serif`, `system-ui`, `sans-serif`), so text still appears in whatever the device provides. The layout shifts; nothing disappears.

11 How to run it

Nothing to install, nothing to start. Two ways work, and only one looks right:

```
1 cd new-design-reference
2 python3 -m http.server 8000
3 # then open http://localhost:8000/ in a browser
```

Listing 50: The way that works properly

Opening the file directly instead renders the page completely unstyled: black text on white, everything in one column.

Why opening the file directly looks broken

The page asks for its stylesheet at `/css/site.css`. The leading slash means “from the very top of this site”. Opened off a disk, the top is the root of your hard drive, so the browser looks for `/css/site.css` there and finds nothing.

Writing `css/site.css` without the slash would have worked both ways. The slashed form was chosen, correct for the live site and inconvenient for local inspection. This one character is the most likely thing to make a newcomer think the project is broken when it is not.

11.1 What you should see

A two-column hero filling the window: on the left, an orange gradient with a giant letter **S** and the words “Drop Salman’s photo here”, with three small frosted badges drifting gently over it; on the right, the name “Salman Adnan” with the surname in orange, a tagline, two buttons and three small statistics. Below that, in order: six project cards, four numbers that count up as they arrive, two large case-study cards, a booking panel and a one-line footer.

12 The agent-facing surface

Several projects in this workspace ship their own Claude Code commands, skills and hooks. **This project ships none.** Checked rather than assumed:

```
1 $ find . -maxdepth 2 \( -name 'CLAUDE.md' -o -name 'skills.md' \)
2 (no output)
3 $ find .claude -type f 2>/dev/null
4 (no output)
```

Listing 51: The check, and its result

No README, no CLAUDE.md, and the important fact lives in a commit message

Somebody cloning this repository is given six files and no statement of what it is, whether it is live, or why everything sits inside a folder called `new-design-reference`. The only explanation anywhere is the message on commit `0dd839e`, invisible unless you know to run `git log`. For a repository whose central fact is “this is parked, the live site is elsewhere”, that belongs in a README where it cannot be missed. The same gap exists in the sibling `ai-digitalise`, so it is a habit rather than an oversight, and it would take five minutes to close in both.

The two PDFs in `docs/explainer/` are produced by a workspace-level command, `/explain-projects`, which lives outside this repository and owns only that folder.

13 Deployment

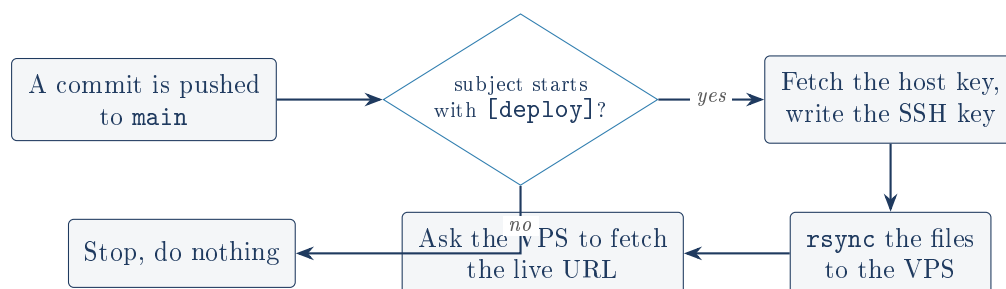


Figure 7: The publishing pipeline as written. Drawn for reference: it is switched off, twice over, and has been since 2026-07-29.

Section 7.6 walks it line by line. In short: a push to `main` checks whether the commit’s first line begins with the literal text `[deploy]`. If so, the files are copied to the VPS with `rsync` and the workflow then asks the VPS to fetch the live address to confirm the site answers. If not, nothing happens.

The `[deploy]` marker must start the subject line

Every repository in this workspace shares this convention. Writing “fix the header and `[deploy]`” does nothing at all: the check is `"$SUBJECT" == "[deploy]"`, which tests the *beginning* of the string.

13.1 What would happen if it were restored

The `rsync` command copies `./`, the repository root, to `/root/salmanadnan.com/` on the server. When it was written, `index.html` was at that root; since `0dd839e` it is one level down. So a restored workflow would copy a folder called `new-design-reference` to the server, and because `-delete` is

set it would remove the real site’s files in the same operation. The live `salmanadnan.com` would go down.

The same commit that moved the files disabled the workflow, so the two changes are consistent. The trap exists only for somebody who re-enables one without reading the other.

An exclusion is stale as well

The `rsync` command excludes `assets/ASSETS.md` so the internal checklist is not published. After the move that file is at `new-design-reference/assets/ASSETS.md`, which the pattern no longer matches, so a restored workflow would publish the checklist to the public internet.

14 Honest findings

Each is argued at the point in the walkthrough where it arises. Collected here so it can be read as a list.

14.1 The design was never finished

1. **The hero photo is a visible placeholder.** Half the opening screen reads “Drop Salman’s photo here / `assets/salman.jpg`” over a gradient. On a personal brand page, whose entire proposition is a person, the missing element is the person. This is the finding that matters most.
2. **None of the eleven assets in `ASSETS.md` exist.** The `assets/` folder contains only the checklist. No photograph, no project thumbnails, no case-study screenshots, no favicon and no social preview image.
3. **Six “UPDATE” comments and one “PLACEHOLDER” comment remain in `index.html`,** some describing work that was done and some describing work that was not.

14.2 The rebrand to orange was left half done

Five places still use the agency blue on an orange-branded site

Commit `05210af` is titled “Differentiate to orange brand”. The token block was duly changed: `-accent` became `#ff6719`. But five rules further down still hard-code the agency blue `rgba(25,103,255,...)` rather than reading the token:

Line	Rule	What a visitor sees
65	<code>.btn-primary:hover</code>	an orange button casting a blue shadow
255–256	<code>.hero-tag</code>	a blue-tinted pill containing orange text
337	<code>.work-card:hover</code>	a project card outlined in blue on hover
435	<code>.case-card:hover</code>	the same on the case-study cards
498	<code>.book-wrap::before</code>	a blue glow above the booking panel

The blue shadow under the orange button (line 65) is the most visible: the two colours are close to complementary, so it reads as a mistake rather than a choice.

None of these is a crash and all five are one-line fixes. Together they are a clean illustration of what the token system is for and what happens when a colour is written literally instead: the rebrand changed the tokens and could not reach the literals.

14.3 Dead code

4. **Two stylesheet rules for the reversed case-study card have never applied.** `site.css` lines 436 and 437 target `.case-card.reverse`; `index.html` line 321 writes `class="case-card case-card-reverse"`. A hyphen is not a second class, so the selector never matches. A third rule in the responsive block (line 559) is dead for the same reason. The intended layout happens anyway, by inline `order` properties on lines 322 and 340, so nothing looks wrong and three rules do nothing.
5. **Four tokens are defined and never read:** `-s2`, `-accent2`, `-rsm` and `-shadow`. Counted mechanically; each returns zero uses. `-accent2` is the agency blue, kept as a secondary colour and then never used as one.
6. **The class counted is added by the JavaScript and styled nowhere.**
7. **`netlify.toml` configures a host that is not used.** The deploy path is `rsync` to a VPS running Caddy; Netlify appears nowhere else.

14.4 Content and consistency

8. **The three hero statistics duplicate three of the four in the metrics band,** written as plain text in one place and as data attributes in the other. So the same numbers exist twice in two formats, and `ASSETS.md` line 24 knows it: it tells the reader to update both.
9. **Two of the three instructions in `ASSETS.md` are stale.** It says to search for `YOUR_CAL_LINK`, which no longer exists, and refers to three `<!-- UPDATE -->` comments in the work grid and case sections, which do not exist either.
10. **The six project cards show a play button on hover and play nothing.** Unlike the sibling repository, these cards do not set `cursor: pointer`, so the promise is weaker here.

14.5 Accessibility

11. **The counting numbers ignore the reduced-motion preference.** The stylesheet honours it carefully, and goes further than the sibling by explicitly stopping the floating badges. The counter is animated in JavaScript over 2.2 seconds and never checks. The fix is four lines, given in section 7.3.
12. **Neither button has a focus state,** so keyboard navigation relies on whatever outline the browser draws, often nearly invisible on a dark background. These are the page's only interactive elements.
13. **The metric values are invisible without JavaScript.** The real numbers live in data attributes and the visible text is 0, so a search engine or a visitor with JavaScript disabled sees a page claiming zero of everything.
14. **The navigation logo is not a link.** It is a `<span>`, so there is no way back to the top of the page by clicking it. On a single-page site this is defensible and it differs from the sibling, where the logo is a link.

15 The project's story

Parts 1 to 4 and part 8 are fact, taken from the repository and its history. Parts 5, 6 and 7 are advice and judgement, labelled where they begin.

15.1 Deliverables

- A complete, self-contained one-page personal brand website: 395 lines of HTML, 575 of CSS, 64 of JavaScript, no dependencies to install.
- A hosting configuration for Netlify setting four security response headers.
- A GitHub Actions workflow publishing to the agency VPS over SSH, gated on a `[deploy]` marker, with retries on both the host-key fetch and the file copy, and a post-deploy check that the site answers.
- An asset checklist naming the eleven files needed to finish the design.

What it did not produce is a live site. `salmanadnan.com` serves a different build.

15.2 Timeline and effort

Ten commits across two calendar days.

Date	Commit	Subject
2026-07-28	1f7ed12	Launch salmanadnan.com, a personal brand site
2026-07-28	014406e	Update booking link to <code>cal.com/salmanadnan/meeting</code>
2026-07-28	05210af	<code>[deploy]</code> Differentiate to orange brand, add deploy workflow
2026-07-28	692e162	<code>[deploy]</code> Replace all placeholder content with real Daztify project data, orange theme, VPS <code>rsync</code> deploy
2026-07-29	066917a	<code>[deploy]</code> Trigger deploy after VPS secrets added
2026-07-29	cfa65d7	<code>[deploy]</code> Fix VPS deploy key
2026-07-29	431ebbf	<code>[deploy]</code> Fix <code>/root</code> traverse permission for deploy user
2026-07-29	6b1aa25	<code>[deploy]</code> Retry after transient keyscan failure
2026-07-29	6fcffee	<code>[deploy]</code> Retry, runner IP lottery
2026-07-29	0dd839e	Move new design into <code>new-design-reference</code> and disable auto-deploy

Table 8: The complete commit history. Subjects are reproduced as written except that `1f7ed12` and `6fcffee` use a long dash where this table uses a comma; that character is not printable in this document's typeface.

The shape is identical to the sibling's and slightly worse: four commits on day one building the thing, then five consecutive commits on day two fighting the deploy, then one commit switching it off. Nothing in the history suggests the design was difficult. Everything suggests publishing it was.

This is calendar time, not effort. Git records when commits happened, not hours spent, and no document here records effort, so none is claimed.

15.3 Challenges

Each is claimed because the repository carries a scar for it.

1. SSH access to the VPS, across three commits. `066917a` (the workflow existed before the secrets did), `cfa65d7` (the key was wrong), and `431ebbf` (the key was right but the `deploy` user could not traverse into `/root` to reach the target folder). The last is a genuinely unobvious Unix permission problem and its workaround survives in three `rsync` flags.

2. GitHub's runners being refused, across two more commits. 6b1aa25 is titled "Retry after transient keyscan failure" and 6fcffee "Retry, runner IP lottery". That second title is the diagnosis stated outright: GitHub's runners come from a wide pool of addresses and some were being refused. This repository is where the evidence lives; the sibling has the same two retry loops and no commit explaining them.

3. Deciding what the brand was. Commit 05210af exists solely to differentiate this site from the agency's blue. As section 14.2 shows, the change reached the tokens and missed five literals, which is the trace of a decision made after the design was written rather than before.

4. Being overtaken. The final commit is a retreat, not a fix. The page was finished and working when something else became the live site. That is the largest event recorded here and it was not technical.

15.4 Outcomes

- **Built, and unfinished.** It renders and is responsive, and the single most prominent element on the page is still a placeholder asking for a photograph.
- **Not live, deliberately.** Parked under `new-design-reference/` with the workflow disabled twice over.
- **Superseded.** salmanadnan.com answers from the same VPS with a different build that has a proper title, a correct canonical link and a social preview image, none of which this version has.
- **One thing did survive.** The live site uses the same booking link, `cal.com/salman-adnan/meeting`, set here in commit 014406e.
- **Dormant** since 2026-07-29.

15.5 What to do (judgement)

From here on this is advice, not a record of project decisions.

- **Read git log before touching anything.** The only statement of this repository's status lives in a commit message.
- **Serve the folder over HTTP to view it,** never open the file directly.
- **Read it beside ai-digitalise.** The two are the same skeleton with different content, and the differences between them are where the interesting decisions are.
- **Keep the [deploy] convention and both retry loops** if the workflow is ever restored. They were paid for over five commits.
- **Treat the hero as the reusable asset.** The two-column layout with fading edges and drifting frosted badges is the best-designed thing in either sibling, and it is about eighty lines of CSS.

15.6 What not to do (judgement)

- **Do not re-enable the workflow without changing the rsync source path.** It would copy a folder rather than a page, with `-delete` set, and take the live salmanadnan.com down.
- **Do not publish this design without the photograph.** The placeholder is the largest element on the page and it is addressed to the developer.
- **Do not trust ASSETS.md.** Two of its three instructions refer to things that are no longer in the code, or never were.

- **Do not fix the five blue literals by changing the token.** The token is correct; the literals are wrong. Changing `-accent` back to blue would make five rules consistent and the entire site wrong.
- **Do not assume this repository is what visitors see.** It is not.

15.7 If you were starting over (judgement)

Entirely opinion, argued from what the code shows.

Keep: the no-build-step decision, the token block, the three-behaviour JavaScript, the reduced-motion block (which is better here than in the sibling, since it also stops the floating badges), and the two-column hero. For a page this size every one of those is right, and a framework would have added a hundred times the machinery for nothing a visitor could perceive.

Do differently: get the photograph first, because a personal brand page without a photograph of the person is not a draft of the design, it is a draft of the idea. Write a README on day one. Use a relative stylesheet path. Never write a colour literally when a token exists, which would have made the rebrand a one-line change instead of a six-line one with five lines missed. Write the real numbers in the HTML and let the counter animate towards them, so the page never claims zero. And state the relationship to `ai-digitalise` in both repositories.

Drop: `netlify.toml`, and the four unused tokens.

Built today it would look almost identical. The only structural change worth making is moving the four security headers out of `netlify.toml` and into the Caddy configuration on the VPS, where they would actually take effect.

15.8 Open threads

- **Eleven assets never delivered**, the photograph above all.
- **Five blue literals** on an orange-branded site.
- **Three dead stylesheet rules** for a class the HTML does not use.
- **Seven stale comments** in `index.html`.
- **The live `ai.digitalise.agency` canonical link** points at this domain, which is a bug in the portfolio repository that produced both live sites, and is recorded in this document because this is where the evidence for it was found.
- **The relationship to `ai-digitalise` is undocumented** in both repositories.

16 Glossary

ARIA

Attributes beginning `aria-` describing a page to screen readers. They change nothing visually.

Attribute

Extra information inside an opening tag, as a name and a value.

Breakpoint

A screen width at which the layout changes. This project has two: 960 and 600 pixels.

Canonical URL

A declaration of where content authoritatively lives, used by search engines to avoid indexing it twice.

Class

A label attached to elements so CSS and JavaScript can find them.

clamp

A CSS function taking a minimum, a preferred value and a maximum.

Commit

One recorded change to a repository, with a message explaining it.

CSS

Cascading Style Sheets: the file controlling appearance.

Custom property

A named value in CSS, defined `-name` and read with `var(-name)`.

Data attribute

An attribute beginning `data-` that HTML ignores and JavaScript can read.

Element

One thing on the page, produced by a tag.

Flexbox

A layout system arranging children in a row or a column.

Grid

A layout system arranging children in rows and columns at once.

Hex colour

A colour written as six characters after a hash.

HTML

HyperText Markup Language: content and structure.

IntersectionObserver

A browser facility reporting when an element scrolls into view.

JavaScript

The language running in the browser after the page loads.

Media query

CSS applying only when a condition about the screen is true.

order

A CSS property setting the position of an item inside a flex or grid container, independent of its position in the HTML.

prefers-reduced-motion

A system setting asking pages to avoid animation.

Pseudo-element

An extra box CSS creates with no tag in the HTML, written `::before` or `::after`.

rsync

A program copying files between computers, transferring only what differs.

Selector

The part of a CSS rule saying which elements it applies to.

SSH

Secure Shell: the standard way to connect to another computer securely.

SVG

A picture described as shapes and coordinates rather than pixels.

Tag

The marker creating an element, written between angle brackets.

Token

In this project, one of the nineteen named values at the top of the stylesheet.

Viewport

The visible area of the page in the browser window.

VPS

Virtual Private Server: a rented computer controlled remotely.

Web server

A program answering requests for files over the network.

Workflow

A file describing a job GitHub runs automatically.

A Appendix A: the file ledger

Every path git tracks, with its tier, its line count, and the section that explains it. This is what makes the coverage claim checkable rather than merely asserted: run `git ls-files` and compare.

`.github/`

Path	Coverage	Lines	Section
<code>.github/workflows-disabled/deploy.yml.disabled</code>	explained in full	103	7.6

Table 9: Coverage ledger: `.github/`. 1 file, 103 lines.

`new-design-reference/`

Path	Coverage	Lines	Section
<code>new-design-reference/index.html</code>	explained in full	395	7.1
<code>new-design-reference/css/site.css</code>	explained in full	575	7.2
<code>new-design-reference/js/main.js</code>	explained in full	64	7.3
<code>new-design-reference/netlify.toml</code>	explained in full	11	7.4
<code>new-design-reference/assets/ASSETS.md</code>	explained in full	32	7.5

Table 10: Coverage ledger: `new-design-reference/`. 5 files, 1,077 lines.

Untracked directories (tier 3)

There are none. `git status -porcelain -ignored` produces no output at all: no ignored files, no build output, no dependency folder, and not even a `.gitignore`. Every file on disk is tracked and every tracked file is explained above.

The arithmetic

`git ls-files` lists **6** paths. All **6** are tier 1, explained in full, line by line, in section 7. **0** files are tier 2: there is no generated, vendored, minified or binary content, checked with a search for generator banners that returned nothing. **0** directories are tier 3, because nothing on disk is untracked. Total lines accounted for: **1,180**, the sum reported by `git ls-files -z | xargs -0 wc -l`.

$6 + 0 = 6$, and $103 + 1,077 = 1,180$. The ledger reconciles.